🧠 AI Evolution Program

Parte 14 — Frontera, investigación y proyectos integradores

Integra todo el programa y mantiene una zona explícita para temas emergentes, distinguiendo evidencia consolidada de prototipos y expectativas.

1. 172 — IA neuro-simbólica
2. 173 — Causal AI y descubrimiento científico
3. 174 — World models y simulación interna
4. 175 — Razonamiento y cómputo en tiempo de inferencia
5. 176 — Aprendizaje continuo y adaptación
6. 177 — Privacidad diferencial y aprendizaje federado
7. 178 — IA para programación y modernización
8. 179 — IA para ciberseguridad y defensa
9. 180 — IA para educación y aprendizaje adaptativo
10. 181 — IA para ciencia, clima y salud responsable
11. 182 — Cómo vigilar la frontera sin perseguir modas
12. 183 — Inferencia privada, computación confidencial y attestation
13. 184 — Capstone final: sistema de IA evolutivo

172 — IA neuro-simbólica

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `frontier` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ia neuro-simbólica** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ia neuro-simbólica usando los conceptos `neuro-symbolic`, `logic`, `neural`, `reasoning`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`neuro-symbolic`, `logic`, `neural`, `reasoning`

🗺️ Ubicación en el mapa de la IA

La IA neuro-simbólica intenta cerrar el ciclo histórico del programa: une la IA simbólica de las Partes 1-2 (lógica, búsqueda, planificación) con el aprendizaje profundo de las Partes 4-6 (representaciones aprendidas desde datos). Ninguna de las dos tradiciones resolvió sola el problema completo: las redes perciben pero no garantizan consistencia lógica; los sistemas simbólicos razonan pero no perciben. Esta clase abre la Parte 14 porque los sistemas híbridos —LLM + solver, agente + verificador formal— son hoy una de las direcciones de frontera más activas y ya los usaste sin nombrarlos: un agente que llama a una calculadora o a un motor de reglas es, según la taxonomía de Kautz, un sistema Neuro[Symbolic].

📖 Fundamentos

🧱 Qué aporta cada tradición

Capacidad	Simbólico (lógica, reglas)	Neural (aprendizaje profundo)
Percepción (imagen, audio, texto crudo)	Débil: requiere features a mano	Fuerte: aprende representaciones
Razonamiento composicional exacto	Fuerte: deducción correcta por construcción	Débil: errores en cadenas largas
Garantías y verificabilidad	Prueba formal inspeccionable	Solo evaluación empírica
Datos necesarios	Pocos (conocimiento experto)	Muchos ejemplos
Robustez ante ruido	Frágil (matching exacto)	Tolerante (interpolación)

La tesis neuro-simbólica: **usar la red donde el problema es percepción y el símbolo donde el problema es deducción**, con una interfaz explícita entre ambos.

🗂️ Taxonomía de Kautz (2020-2022)

Henry Kautz propuso en su conferencia AAAI 2020 (publicada en *AI Magazine*, 2022) seis patrones de integración que siguen siendo el vocabulario estándar:

```
1. Symbolic Neuro symbolic   entrada y salida simbólicas, red en el medio.
                             Ej.: un LLM que recibe texto y produce texto.
2. Symbolic[Neuro]           sistema simbólico que invoca una subrutina neural.
                             Ej.: AlphaGo — búsqueda MCTS (simbólica) que consulta
                             una red para evaluar posiciones.
3. Neuro | Symbolic          red y razonador cooperan como co-rutinas acopladas.
                             Ej.: Neuro-Symbolic Concept Learner (percepción
                             neural + programa simbólico sobre la escena).
4. Neuro: Symbolic -> Neuro  conocimiento simbólico compilado en el entrenamiento
                             de la red (reglas como pérdida o restricción).
5. Neuro_{Symbolic}          el razonamiento simbólico se tensoriza dentro de la
                             red. Ej.: Logic Tensor Networks.
6. Neuro[Symbolic]           una red que decide invocar un motor simbólico.
                             Ej.: un LLM agéntico que llama a un solver, a una
                             calculadora o a un verificador de pruebas.
```

⚙️ Mecanismo típico: percepción probabilística + restricción lógica

El patrón que implementan sistemas como DeepProbLog (Manhaeve et al., 2018) es:

```
1. La red neuronal produce distribuciones sobre símbolos:
   P(simbolo_i | entrada_i)   — percepción incierta.
2. Un programa lógico define qué combinaciones de símbolos son válidas
   o qué se deduce de ellas (regla dura, sin aprender).
3. La probabilidad de una conclusión = suma de probabilidades de todas
   las asignaciones de símbolos que la hacen verdadera
   (weighted model counting).
4. El gradiente atraviesa el paso lógico, así la restricción simbólica
   supervisa a la red sin etiquetas intermedias.
```

Ejemplos actuales del patrón híbrido: **AlphaGeometry** (Trinh et al., *Nature* 2024) alterna un modelo de lenguaje que propone construcciones auxiliares con un motor de deducción simbólica que verifica; resolvió 25 de 30 problemas de geometría de olimpiada. **PAL / program-aided LMs**: el LLM escribe un programa y un

intérprete de Python lo ejecuta — la aritmética la garantiza el intérprete, no la red. Tu propio uso de *tool calling* en la Parte 9 es la forma industrial del tipo 6.

🔍 Conexión con el laboratorio

`run_lab("frontier")` no entrena una red: aplica una **regla simbólica dura** sobre afirmaciones de frontera (si no hay evidencia → `unverified` → fuera del currículo). Ilustra el valor del componente simbólico: la decisión es inspeccionable, correcta por construcción y no depende de datos de entrenamiento.

🧮 Ejemplo trabajado

Percepción neural incierta + regla simbólica, al estilo DeepProbLog, a mano.

Dos imágenes de dígitos pasan por un clasificador:

```
Imagen A: P(3) = 0.7   P(5) = 0.3
Imagen B: P(2) = 0.6   P(7) = 0.4
```

Sin restricción, la lectura más probable es (3, 2) con 0.7 × 0.6 = 0.42. Ahora el conocimiento simbólico dice: "la suma es 10" (regla del dominio, cierta).

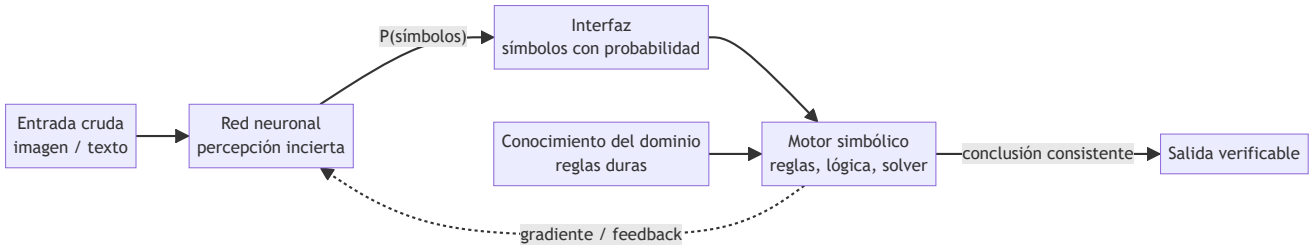
Probabilidad de cada mundo (asumiendo independencia):

```
(3,2) -> suma 5    p = 0.7 × 0.6 = 0.42   viola la regla
(3,7) -> suma 10   p = 0.7 × 0.4 = 0.28   cumple
(5,2) -> suma 7    p = 0.3 × 0.6 = 0.18   viola
(5,7) -> suma 12   p = 0.3 × 0.4 = 0.12   viola
```

Condicionando a la regla (renormalizar sobre los mundos válidos): P((3,7) | suma=10) = 0.28 / 0.28 = **1.0**. La regla simbólica corrigió la lectura neural: la hipótesis globalmente más probable (3,2) era inconsistente. Con más de un mundo válido, la renormalización repartiría la masa entre ellos — así el componente lógico convierte percepción ruidosa en conclusiones coherentes.

📊 Propiedades y comparación

Enfoque	Percepción	Garantía deductiva	Datos	Ejemplo actual
Solo simbólico	Manual	Sí (prueba)	Mínimos	Solvers SAT/SMT
Solo neural	Aprendida	No	Masivos	LLM puro
Symbolic[Neuro]	Delegada a la red	En el marco simbólico	Medios	AlphaGo
Neuro[Symbolic]	En la red	En el motor invocado	Masivos + motor	LLM + solver, PAL
Compilación (tipo 4-5)	Aprendida	Aproximada (soft constraint)	Menos que puro neural	Logic Tensor Networks



⚠️ Errores conceptuales frecuentes

1. **"Neuro-simbólico = poner un if después de la red"**. El punto es la interfaz probabilística: la incertidumbre de la red debe propagarse al razonador (model counting, renormalización), no descartarse con un argmax prematuro.
2. **"Los LLM ya razonan, lo simbólico es obsoleto"**. Los LLM fallan en cadenas deductivas largas y aritmética exacta; AlphaGeometry y el tool calling existen precisamente porque delegar la deducción a un motor exacto mejora el resultado.
3. **"Lo simbólico no escala"**. Los solvers SAT/SMT modernos resuelven instancias industriales con millones de cláusulas; lo que no escala es codificar percepción a mano — por eso la división de trabajo.
4. **"La taxonomía de Kautz es exclusiva"**. Un sistema real combina varios tipos: un agente LLM (tipo 1 en su núcleo) que llama a un verificador (tipo 6) dentro de un orquestador con reglas (tipo 2).
5. **"La regla dura siempre gana"**. Si la regla del dominio es errónea o la percepción está mal calibrada, condicionar sobre ella amplifica el error con total confianza — la calidad del conocimiento simbólico es un supuesto crítico.

🚀 Del aprendizaje a la operación

Entre este núcleo y un sistema real faltan: entrenar de verdad la parte neural con la pérdida que atraviesa la lógica (diferenciación sobre model counting, costosa y con técnicas propias); ingeniería del conocimiento para escribir y mantener las reglas del dominio; decidir qué hace el sistema cuando la percepción y la regla se contradicen con alta confianza (¿abstenerse, escalar a humano?); y evaluación separada de cada componente además del sistema conjunto, porque un error puede originarse en la red, en la regla o en la interfaz.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("frontier")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;

- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kautz, H. (2022). *The Third AI Summer: AAAI Robert S. Engelmore Memorial Lecture*. AI Magazine, 43(1). doi:10.1002/aaai.12036 — uso: fuente primaria del mecanismo estudiado
- Manhaeve, R. et al. (2018). *DeepProbLog: Neural Probabilistic Logic Programming*. NeurIPS 2018. arXiv:1805.10872 — uso: fuente primaria del mecanismo estudiado
- Mao, J. et al. (2019). *The Neuro-Symbolic Concept Learner*. ICLR 2019. arXiv:1904.12584 — uso: fuente primaria del mecanismo estudiado
- Trinh, T. et al. (2024). *Solving olympiad geometry without human demonstrations* (AlphaGeometry). Nature 625. doi:10.1038/s41586-023-06747-5 — uso: fuente primaria del mecanismo estudiado
- Garcez, A. & Lamb, L. (2020). *Neurosymbolic AI: The 3rd Wave*. arXiv:2012.05876 — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach* (4.ª ed.), caps. 7-10 (lógica) como base simbólica. Sitio oficial — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P27 · Dominar el go con redes neuronales profundas y búsqueda en árbol	2016	Une las dos tradiciones de la IA: la búsqueda simbólica de la parte 01 y el aprendizaje profundo de la parte 04, en un solo sistema.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · caps. 7-10 · obra de referencia de la parte 14
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define ia neuro-simbólica sin usar una marca o framework como definición.
2. Explica la relación entre neuro-symbolic, logic, neural, reasoning.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

173 — Causal AI y descubrimiento científico

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **causal ai y descubrimiento científico** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar causal ai y descubrimiento científico usando los conceptos `causal`, `interventions`, `science`, `discovery`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`causal`, `interventions`, `science`, `discovery`

🗺️ Ubicación en el mapa de la IA

Toda la estadística y el machine learning de las Partes 2-3 operan en el nivel de la **asociación**: P(Y|X), lo que se observa. La IA causal añade los niveles que la correlación no puede alcanzar: predecir el efecto de una **intervención** y razonar sobre **contrafactuales**. Es frontera porque el aprendizaje profundo actual —incluidos los LLM— aprende asociaciones, y las preguntas que importan en ciencia, medicina y política pública ("¿qué pasa si administro el fármaco?") son causales. El descubrimiento científico asistido por IA depende de esta distinción: generar hipótesis causales verificables, no solo patrones.

📖 Fundamentos

🪜 La escalera de la causalidad (Pearl)

Peldaño	Pregunta	Operación	Ejemplo
1. Asociación	¿Qué veo?	P(Y | X)	pacientes que toman X se recuperan más
2. Intervención	¿Qué pasa si hago?	P(Y | do(X))	si *administro* X a todos, ¿cuántos se recuperan?
3. Contrafactual	¿Qué habría pasado?	P(Y_x | X', Y')	este paciente no tomó X y murió; ¿habría vivido con X?

La diferencia central: **condicionar no es intervenir**. P(Y | X=x) filtra la población observada (con todos sus sesgos de selección); P(Y | do(X=x)) describe una población donde X se fija desde fuera, cortando las causas naturales de X. Ningún ajuste de curvas sobre datos observacionales puede, por sí solo, subir un peldaño: hace falta un **modelo causal** (un grafo dirigido acíclico, DAG, con supuestos explícitos) o un experimento aleatorizado.

🔧 do-cálculo y el criterio de puerta trasera

Un DAG causal codifica qué variables causan qué. El problema de identificación: ¿puede escribirse P(Y|do(X)) solo con probabilidades observacionales? El **criterio de puerta trasera** da la respuesta más usada: si un conjunto Z bloquea todos los caminos "de puerta trasera" entre X e Y (caminos con flecha entrando a X) y no contiene descendientes de X, entonces:

```
P(Y | do(X=x)) = Σ_z  P(Y | X=x, Z=z) · P(Z=z)      (fórmula de ajuste)
```

El do-cálculo de Pearl (tres reglas de reescritura sobre el grafo) generaliza esto: es **completo** — si una cantidad causal es identificable desde datos observacionales y el grafo, las reglas la derivan; si no lo es, ninguna cantidad de datos la estima sin más supuestos.

🔬 Descubrimiento causal

Ir de datos al grafo (no del grafo a la estimación). Tres familias:

```
1. Basados en restricciones (PC, FCI): prueban independencias condicionales
   X ⊥ Y | Z en los datos y descartan grafos incompatibles. Devuelven una
   clase de equivalencia, no un grafo único.
2. Basados en puntuación (GES): buscan el DAG que maximiza un score
   (BIC) sobre los datos.
3. Modelos causales funcionales (LiNGAM, mecanismos aditivos): usan
   asimetrías de las distribuciones (no-gaussianidad, no-linealidad) para
   orientar aristas que las independencias dejan ambiguas.
```

Límites duros: sin supuestos (suficiencia causal —no hay confusores ocultos—, fidelidad), los datos observacionales solo identifican el grafo **hasta su clase de equivalencia de Markov**: X→Y y X←Y suelen ser indistinguibles. Las intervenciones (experimentos) rompen la simetría — por eso el laboratorio automatizado (ciclo hipótesis → experimento → actualización) es la forma que toma el descubrimiento científico con IA.

🔍 Conexión con el laboratorio

`run_lab("probability")` calcula un posterior bayesiano: eso es peldaño 1, asociación pura. La clase pide notar qué le falta a ese cálculo para responder una pregunta causal: un grafo, supuestos de intervención, o un experimento.

🧮 Ejemplo trabajado

Confusión clásica. Un tratamiento X, recuperación Y, y gravedad del caso Z que causa ambas (los médicos dan el tratamiento a los graves). Datos observacionales:

```
P(Z=grave) = 0.5
P(X=1 | grave) = 0.8      P(X=1 | leve) = 0.2
P(Y=1 | X=1, grave) = 0.6   P(Y=1 | X=0, grave) = 0.4
P(Y=1 | X=1, leve)  = 0.9   P(Y=1 | X=0, leve)  = 0.8
```

Asociación (peldaño 1): P(Y=1|X=1) mezcla graves y leves según quién recibió el tratamiento. P(grave|X=1) = 0.8·0.5 / (0.8·0.5 + 0.2·0.5) = 0.8. Entonces P(Y=1|X=1) = 0.8·0.6 + 0.2·0.9 = **0.66**. Análogo: P(grave|X=0) = 0.2, y P(Y=1|X=0) = 0.2·0.4 + 0.8·0.8 = **0.72**. ¡El tratamiento parece dañino! (0.66 < 0.72).

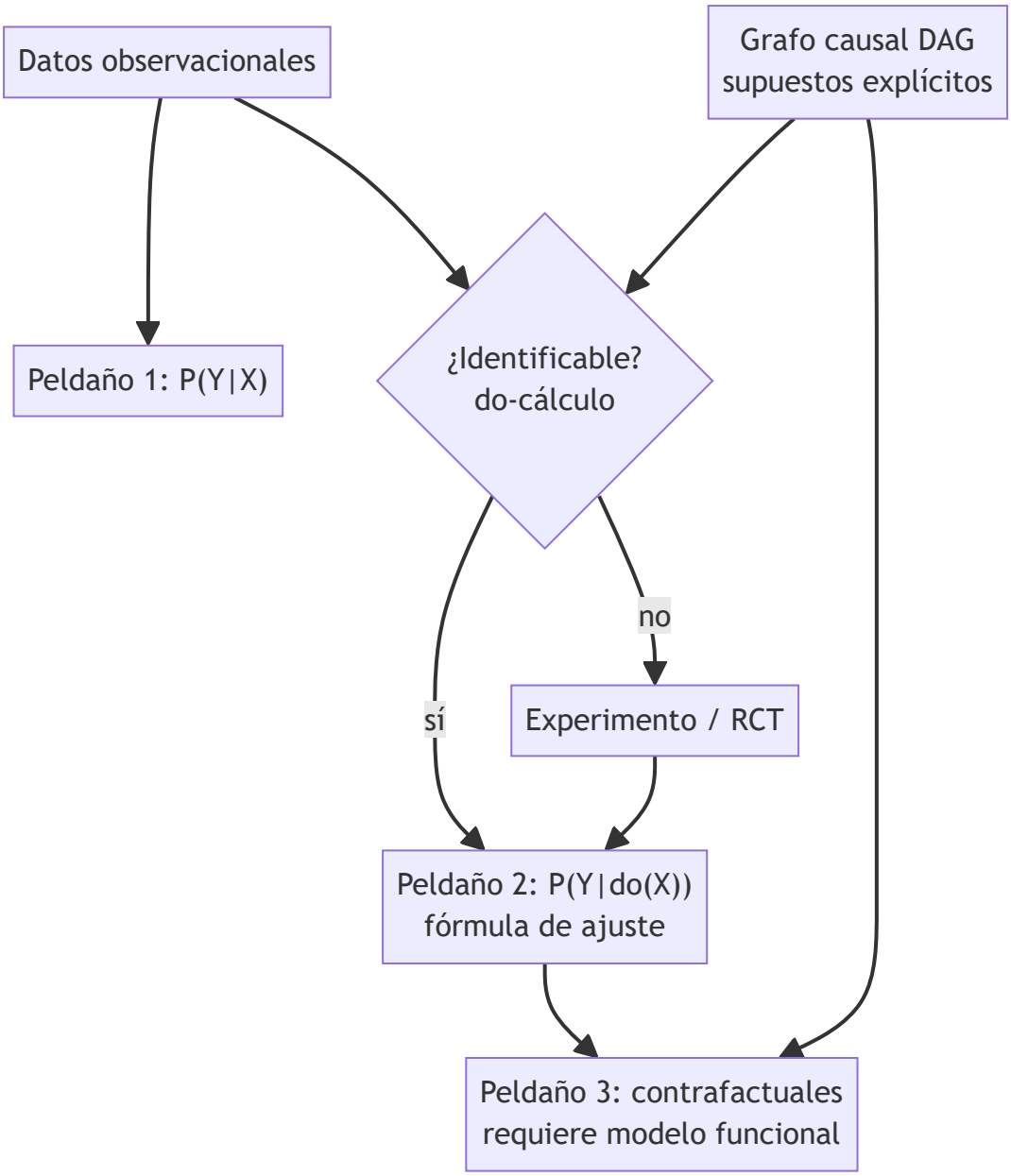
Intervención (peldaño 2): Z satisface la puerta trasera (Z→X, Z→Y). Ajuste:

```
P(Y=1 | do(X=1)) = 0.6·0.5 + 0.9·0.5 = 0.75
P(Y=1 | do(X=0)) = 0.4·0.5 + 0.8·0.5 = 0.60
```

El efecto causal es **+0.15 a favor del tratamiento**. La asociación tenía el signo invertido porque los graves recibían más tratamiento (paradoja de Simpson). Mismos datos, conclusión opuesta: lo que cambió fue el modelo causal, no los números.

📊 Propiedades y comparación

Enfoque	Pregunta que responde	Requiere	Riesgo principal
Correlación / ML predictivo	P(Y|X)	Solo datos	Confundir predicción con efecto
Ajuste por puerta trasera	P(Y|do(X))	DAG correcto + Z observado	Grafo mal especificado
Experimento aleatorizado (RCT)	P(Y|do(X))	Poder intervenir	Coste, ética, validez externa
Descubrimiento causal (PC/GES)	Estructura del grafo	Independencias fieles, sin confusores ocultos	Clase de equivalencia, no grafo único
Contrafactuales (SCM)	P(Y_x|evidencia)	Modelo funcional completo	Supuestos no verificables



⚠️ Errores conceptuales frecuentes

1. **"Con suficientes datos, la correlación se vuelve causalidad"**. No: la identificación causal es un problema de supuestos, no de tamaño muestral. El ejemplo trabajado invierte el signo con probabilidades exactas (n = ∞).
2. **"Condicionar por todo lo posible siempre ayuda"**. Condicionar por un **colisionador** (X→W←Y) o un descendiente de X *abre* sesgos que no existían. El criterio de puerta trasera dice qué ajustar y qué no.
3. **"El DAG se aprende de los datos y listo"**. Los algoritmos de descubrimiento devuelven clases de equivalencia bajo supuestos fuertes (sin confusores ocultos); el grafo final incorpora conocimiento del dominio y es refutable.
4. **"do(X) es lo mismo que ver X"**. do(X) borra las flechas que llegan a X; condicionar las conserva. La paradoja de Simpson vive en esa diferencia.

5. **"Los contrafactuales son verificables"**. El peldaño 3 nunca es observable directamente (no se puede rebobinar al paciente); se apoya en un modelo funcional cuyos supuestos hay que declarar y defender.

🚀 Del aprendizaje a la operación

Para usar esto de verdad faltan: construir el DAG con expertos del dominio y someterlo a refutación (pruebas de independencia, análisis de sensibilidad ante confusores no observados); estimadores robustos con muestras finitas (ponderación por propensity, doble robustez) en lugar de las probabilidades exactas del ejemplo; y en descubrimiento científico, cerrar el ciclo con experimentos reales —la IA propone el grafo y el experimento que mejor lo discrimina, el laboratorio lo ejecuta— con registro de versiones de hipótesis igual que se versiona código.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Pearl, J. & Mackenzie, D. (2018). *The Book of Why: The New Science of Cause and Effect*. Basic Books. — uso: desarrollo extendido del tema
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2.ª ed.). Cambridge University Press. — uso: referencia consultada en su fuente original
- Pearl, J. (2009). *Causal inference in statistics: An overview*. Statistics Surveys 3. doi:10.1214/09-SS057 — uso: fuente primaria del mecanismo estudiado
- Peters, J., Janzing, D. & Schölkopf, B. (2017). *Elements of Causal Inference*. MIT Press (acceso abierto). Página oficial — uso: referencia consultada en su fuente original
- Schölkopf, B. et al. (2021). *Toward Causal Representation Learning*. Proceedings of the IEEE. arXiv:2102.11107 — uso: fuente primaria del mecanismo estudiado
- Spirtes, P., Glymour, C. & Scheines, R. (2000). *Causation, Prediction, and Search* (2.ª ed.). MIT Press. — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P47 · Predicción de estructura de proteínas de alta precisión con AlphaFold	2021	Resuelve en la práctica un problema abierto de cincuenta años en biología, y demuestra que la IA puede producir conocimiento científico, no solo productos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Spirtes, P., Glymour, C. y Scheines, R. — *Causation, Prediction, and Search*	2000	ISBN 9781461227489	citada en las referencias de esta clase
Peters, Jonas, Janzing, Dominik y Schölkopf, Bernhard — *Elements of Causal Inference*	2017	ISBN 9780262037310 · web de la obra	citada en las referencias de esta clase
Pearl, J. y Mackenzie, D. — *The Book of Why*	2018	ISBN 9780465097609	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define causal ai y descubrimiento científico sin usar una marca o framework como definición.
2. Explica la relación entre causal, interventions, science, discovery.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

174 — World models y simulación interna

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `frontier` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **world models y simulación interna** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar world models y simulación interna usando los conceptos `world models`, `simulation`, `planning`, `latent`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`world models`, `simulation`, `planning`, `latent`

🗺️ Ubicación en el mapa de la IA

En la Parte 2 viste agentes que planifican sobre modelos dados (MDP conocidos) y en RL model-free el agente aprende sin modelo, pagando en muestras. Los *world models* cierran esa brecha: el agente **aprende un modelo del entorno y planifica dentro de él** — "sueña" trayectorias en lugar de sufrirlas. Es frontera porque la predicción en espacio latente (JEPA, Dreamer) es una de las apuestas principales para agentes que entiendan consecuencias físicas antes de actuar, y conecta directamente con la robótica de la Parte 11: un robot que simula internamente choca menos en el mundo real.

📖 Fundamentos

🧱 La arquitectura V-M-C (Ha & Schmidhuber, 2018)

World Models (arXiv:1803.10122) descompone el agente en tres módulos:

```
V (Vision):     un autoencoder variacional comprime cada frame o
                observación o_t en un código latente z_t (p. ej. 64 dims
                en vez de 64×64×3 píxeles).
M (Memory):     una RNN con salida de mezcla de gaussianas (MDN-RNN)
                aprende la dinámica en el latente:
                P(z_{t+1} | z_t, a_t, h_t)  — predicción estocástica.
C (Controller): una política diminuta (lineal en el paper) que decide
                a_t = C([z_t, h_t]). Al ser pequeña, puede optimizarse
                con métodos evolutivos (CMA-ES).
```

El resultado célebre: el controlador puede **entrenarse íntegramente dentro del sueño** — trayectorias generadas por M sin tocar el entorno real— y la política transferirse al entorno verdadero. Dos condiciones lo hacen posible: el modelo M captura suficiente dinámica, y se controla la "temperatura" del sueño para que el agente no explote errores del modelo (atajos que solo existen en la imaginación).

🔮 Por qué predecir en latente y no en píxeles

Predecir píxeles futuros obliga a modelar detalles irrelevantes (textura de la hierba) y promedia futuros posibles en imágenes borrosas. La familia **JEPA** (Joint-Embedding Predictive Architecture, LeCun 2022; I-JEPA, arXiv:2301.08243) lleva el principio al extremo: predecir la **representación** de la parte faltante/futura, no su apariencia. La pérdida vive en el espacio de embeddings; lo impredecible o irrelevante puede simplemente no representarse. Dreamer (Hafner et al.) aplica lo mismo a RL: aprende un modelo recurrente de estados latentes (RSSM) y entrena actor y crítico con **imaginación**: rollouts latentes de ~15 pasos. DreamerV3 (arXiv:2301.04104) resuelve más de 150 tareas con hiperparámetros fijos, incluida la obtención de diamantes en Minecraft desde cero.

⚙️ Planificar con el modelo: el ciclo general

```
1. Recolectar experiencia real (o_t, a_t, r_t, o_{t+1}).
2. Ajustar el world model: encoder ϕ: o→z y dinámica
   T: (z_t, a_t) → distribución sobre z_{t+1} (+ recompensa r̂).
3. Imaginar: desde el z actual, desplegar K trayectorias con
   acciones candidatas, SIN tocar el entorno.
4. Elegir el plan con mejor retorno imaginado (o entrenar la
   política sobre esas trayectorias).
5. Ejecutar 1 acción real, observar, y volver a 1 (el error del
   modelo se corrige con re-planificación frecuente).
```

Este es el mismo patrón Dyna de Sutton & Barto (cap. 8), con la diferencia de que el modelo ya no es una tabla sino una red que comprime observaciones de alta dimensión.

⚠️ El talón de Aquiles: explotación del modelo

Un planificador que optimiza dentro del modelo encuentra sus errores: si M predice mal una zona, el plan "óptimo" puede vivir exactamente ahí (model exploitation). Mitigaciones: horizonte corto, penalizar incertidumbre del modelo (ensembles), re-planificar a menudo, y mezclar experiencia real con imaginada.

🧮 Ejemplo trabajado

World model tabular mínimo: un robot en 3 estados latentes {A, B, C}; C da recompensa 1 al entrar, el resto 0. Dinámica aprendida de la experiencia (estocástica):

```
T(z'|z, avanzar):  A→B 0.9, A→A 0.1;  B→C 0.8, B→B 0.2;  C→C 1.0
T(z'|z, saltar):   A→C 0.4, A→A 0.6;  B→C 0.5, B→B 0.5;  C→C 1.0
```

Desde A, horizonte 2, comparar dos planes **por imaginación** (sin entorno real):

Plan 1: [avanzar, avanzar] — llegar a C en t≤2: paso 1: P(B)=0.9, P(A)=0.1. Paso 2: desde B, P(C)=0.8 → 0.9·0.8 = 0.72; desde A, P(C con avanzar)=0 → 0. Retorno esperado = **0.72**.

Plan 2: [saltar, saltar] — P(C en t=1) = 0.4 (recompensa ya asegurada); si no (0.6 en A), P(C en t=2) = 0.4 → 0.6·0.4 = 0.24. Retorno esperado = 0.4 + 0.24 = **0.64**.

El agente elige el plan 1 sin ejecutar nada. Ahora el límite: si la probabilidad real de B→C fuera 0.5 (el modelo la sobreestima en 0.8), el retorno real del plan 1 sería 0.9·0.5 = 0.45 < 0.64: el plan "óptimo imaginado" es peor en el mundo. Eso es explotación del modelo, y por eso se re-planifica tras cada paso.

📊 Propiedades y comparación

Enfoque	Muestras reales	Cómputo	Riesgo específico	Ejemplo
Model-free (DQN, PPO)	Muchas	Medio	Ineficiencia muestral	Atari clásico
World model + imaginación	Pocas	Alto (entrenar modelo)	Explotación del modelo	Dreamer, V-M-C
Planificación con modelo dado	Ninguna (modelo exacto)	Depende del horizonte	El modelo casi nunca es exacto	MDP de la Parte 2
Predicción en píxeles	—	Muy alto	Futuros borrosos, detalle irrelevante	video prediction
Predicción en latente (JEPA)	—	Medio	Colapso de representaciones	I-JEPA, V-JEPA

⚠️ Errores conceptuales frecuentes

1. **"El world model debe predecir el futuro con exactitud"**. Basta con que ordene bien los planes candidatos; JEPA renuncia explícitamente a predecir apariencia y predice representaciones.
2. **"Entrenar en el sueño siempre transfiere"**. Solo si se controla la explotación del modelo (temperatura en Ha & Schmidhuber, ensembles, horizontes cortos); un plan óptimo en un modelo erróneo puede ser pésimo.
3. **"Un generador de video es ya un world model"**. Generar video plausible no implica dinámica consistente con acciones ni física estable a largo plazo; la evidencia debe evaluarse con intervenciones (¿responde bien a acciones?), no con calidad visual.
4. **"Latente = caja negra inútil"**. El latente es inspeccionable: se puede decodificar, medir qué factores captura y detectar cuándo la incertidumbre del modelo crece (señal para no confiar en el plan).

5. **"Model-based siempre gana a model-free"**. Con simuladores baratos y muestras infinitas, model-free sigue siendo competitivo y más simple; la ventaja del modelo aparece cuando las muestras reales son caras o peligrosas.

🚀 Del aprendizaje a la operación

Del ejemplo tabular a un sistema real faltan: aprender el encoder y la dinámica desde observaciones crudas (VAE/RSSM con sus inestabilidades de entrenamiento); cuantificar la incertidumbre del modelo para penalizar planes que la exploten; presupuestar el cómputo de imaginación (K rollouts × horizonte × frecuencia de re-planificación) contra la latencia de control real; y validar la transferencia sueño→realidad con métricas de intervención, no solo con retorno imaginado.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("frontier")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Ha, D. & Schmidhuber, J. (2018). *World Models*. arXiv:1803.10122 — uso: fuente primaria del mecanismo estudiado
- Hafner, D. et al. (2023). *Mastering Diverse Domains through World Models* (DreamerV3). arXiv:2301.04104 — uso: fuente primaria del mecanismo estudiado
- LeCun, Y. (2022). *A Path Towards Autonomous Machine Intelligence*. OpenReview — uso: referencia consultada en su fuente original
- Assran, M. et al. (2023). *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture* (I-JEPA). CVPR 2023. arXiv:2301.08243 — uso: fuente primaria del mecanismo estudiado
- Sutton, R. & Barto, A. (2018). *Reinforcement Learning: An Introduction* (2.ª ed.), cap. 8 (Dyna, planning and learning). PDF oficial — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P147 · Los modelos recurrentes del mundo facilitan la evolución de políticas	2018	Entrena la política **dentro** de un modelo del entorno aprendido, y demuestra que la política resultante funciona en el entorno real.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase · cap. 8
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define world models y simulación interna sin usar una marca o framework como definición.
2. Explica la relación entre world models, simulation, planning, latent.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

175 — Razonamiento y cómputo en tiempo de inferencia

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `frontier` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **razonamiento y cómputo en tiempo de inferencia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar razonamiento y cómputo en tiempo de inferencia usando los conceptos `reasoning`, `test-time compute`, `verification`, `search`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`reasoning`, `test-time compute`, `verification`, `search`

🗺️ Ubicación en el mapa de la IA

Las leyes de escalado de la Parte 6 gobernaban el *entrenamiento*: más datos y parámetros, mejor modelo. Esta clase estudia el segundo eje, descubierto después: escalar el **cómputo en el momento de responder** — pensar más tiempo, muestrear varias soluciones, verificar antes de contestar. Es la base de los modelos razonadores tipo o1/R1 y de patrones que ya usaste en agentes (Parte 9): un agente que planifica, critica y reintenta está gastando test-time compute. La pregunta de frontera: ¿cuándo un token de "pensamiento" rinde más que un parámetro adicional?

📖 Fundamentos

🔗 Chain-of-thought: hacer visible el cómputo intermedio

Wei et al. (2022, arXiv:2201.11903) mostraron que pedir al modelo pasos intermedios ("pensemos paso a paso") mejora drásticamente tareas aritméticas y de razonamiento, y que el efecto **emerge con la escala** (poco o nada en modelos pequeños). Mecánicamente: cada token generado es un paso de cómputo adicional

condicionado en los anteriores; la cadena externaliza estado intermedio que no cabe en una sola pasada. Advertencia empírica posterior: la cadena verbalizada no siempre refleja el proceso interno real (no es una explicación fiel por construcción).

📋 Self-consistency: muestrear y votar

Wang et al. (2022, arXiv:2203.11171) sustituyen la decodificación voraz por:

```
1. Muestrear k cadenas de razonamiento con temperatura > 0.
2. Extraer la respuesta final de cada cadena.
3. Elegir la respuesta más frecuente (voto por mayoría marginal:
   se marginaliza sobre los caminos de razonamiento).
```

Intuición estadística: hay muchas formas de razonar bien que llegan a la misma respuesta, y muchas de razonar mal que llegan a respuestas distintas. Si cada muestra acierta con probabilidad $p > 0.5$ y los errores están repartidos, la mayoría amplifica p (ver ejemplo trabajado). Coste: k× la inferencia.

🎄 Buscar y verificar: best-of-N, árboles y verificadores

Generalizaciones del mismo principio "generar varios candidatos + seleccionar":

- **Best-of-N con verificador**: generar N soluciones y puntuar con un modelo de recompensa (outcome-based ORM o process-based PRM, que puntúa paso a paso).
- **Tree-of-Thoughts** (arXiv:2305.10601): explorar deliberadamente un árbol de pensamientos parciales con retroceso — búsqueda clásica (Parte 1) sobre estados generados por el LLM.
- **Revisión iterativa**: el modelo critica y corrige su propio borrador.

Snell et al. (2024, arXiv:2408.03314) muestran que asignar el cómputo de inferencia de forma **adaptativa a la dificultad** puede superar a un modelo ~14× más grande respondiendo de un tiro: para preguntas fáciles casi no hay ganancia; para intermedias, buscar más compensa; para las muy difíciles por falta de conocimiento, ningún tiempo de pensamiento alcanza.

📈 Escalado tipo o1: entrenar a pensar

Los modelos razonadores (o1 de OpenAI, 2024; DeepSeek-R1, arXiv:2501.12948) cierran el círculo: se entrenan con **RL sobre trazas largas de razonamiento** premiando la respuesta final verificable (matemática, código). Resultado: el rendimiento crece suavemente con el logaritmo del cómputo de inferencia permitido — una "ley de escalado" del pensamiento. R1 mostró además que la conducta de auto-corrección puede emerger con RL puro sobre recompensas verificables. Límite clave: exige dominios donde verificar sea barato y fiable; donde no hay verificador, el beneficio se diluye y aparece el riesgo de *reward hacking* del juez.

🔍 Conexión con el laboratorio

`run_lab("frontier")` separa afirmaciones por madurez. El test-time compute es terreno fértil para afirmaciones infladas ("el modelo razona como un humano"); la disciplina es la misma: pedir la curva cómputo-precisión y el verificador usado antes de aceptar la claim.

🧮 Ejemplo trabajado

Self-consistency a mano. Un modelo resuelve un problema de aritmética con probabilidad p = 0.6 por muestra (independientes, errores repartidos entre respuestas distintas). Muestreamos k = 5 cadenas:

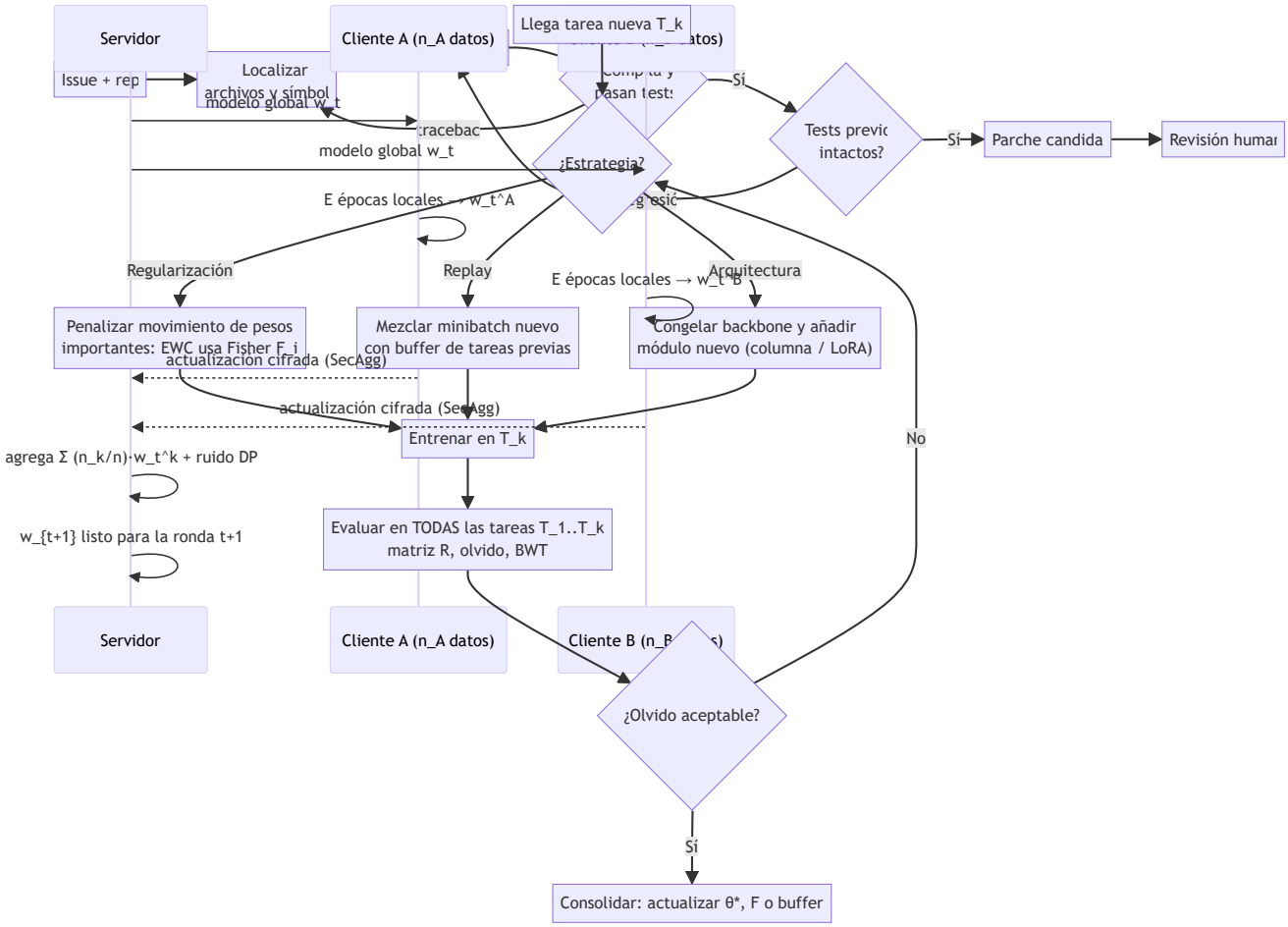
```
P(mayoría correcta) = P(≥3 aciertos de 5)
 = C(5,3)·0.6³·0.4² + C(5,4)·0.6⁴·0.4 + C(5,5)·0.6⁵
 = 10·0.216·0.16  +  5·0.1296·0.4  +  0.0778
 = 0.3456 + 0.2592 + 0.0778 = 0.683
```

Una sola muestra: 60 %. Votando 5: **68.3 %**. Con k = 11: ≈ 75.3 %. La curva crece con rendimientos decrecientes y coste lineal en k.

Contraste crítico: si p = 0.4 (el modelo se equivoca *sistemáticamente* hacia la misma respuesta errónea), la mayoría de 5 acierta solo el 31.7 % — votar **amplifica el sesgo**. Self-consistency mejora cuando los errores son diversos y la señal correcta es el modo; no arregla un modelo sesgado.

📊 Propiedades y comparación

Técnica	Coste (× inferencia)	Requiere	Cuándo gana	Límite
CoT (1 cadena)	~1-3× tokens	Nada extra	Tareas multipaso	Cadena infiel, un solo camino
Self-consistency	k×	Respuesta extraíble	Errores diversos, p > 0.5	Amplifica sesgos sistemáticos
Best-of-N + verificador	N× + juez	Verificador fiable	Dominios verificables	Reward hacking del juez
Tree-of-Thoughts	Variable (poda)	Evaluador de estados	Problemas con estructura de búsqueda	Coste y diseño ad hoc
Modelo razonador (o1/R1)	Tokens de pensamiento	Entrenamiento RL previo	Matemática, código	Dominios sin verificador



⚠️ Errores conceptuales frecuentes

1. **"La cadena de pensamiento explica cómo el modelo llegó a la respuesta"**. Es texto generado, no una traza del cómputo interno; puede racionalizar a posteriori. Para auditoría se necesita más que leer el CoT.
2. **"Votar siempre mejora"**. Solo si los aciertos concentran el modo. Con error sistemático ($p < 0.5$ hacia una misma respuesta), la mayoría empeora la precisión — ver el ejemplo trabajado.
3. **"Más tokens de pensamiento = mejor, siempre"**. La ganancia es logarítmica y depende de la dificultad; en preguntas fáciles se paga latencia por nada, y en las imposibles por falta de conocimiento tampoco ayuda (Snell et al.).
4. **"El verificador es neutral"**. Best-of-N optimiza contra el juez: si el juez tiene sesgos explotables, el sistema selecciona precisamente las respuestas que los explotan.
5. **"o1 demuestra razonamiento general humano"**. Demuestra escalado en dominios con verificación barata (matemática, código); la transferencia a dominios sin verificador es una claim que exige evidencia separada.

🚀 Del aprendizaje a la operación

En producción, el test-time compute es una decisión de **presupuesto**: fijar k/N por consulta según valor y latencia tolerable; enrutar por dificultad estimada (la mayor parte del tráfico no necesita razonar); cachear respuestas verificadas; medir la curva coste-precisión propia en lugar de asumir la de los papers; y vigilar el

modo de fallo silencioso — un sistema que vota o se auto-verifica puede estar amplificando un sesgo sistemático con total confianza.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("frontier")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. arXiv:2201.11903 — uso: fuente primaria del mecanismo estudiado
- Wang, X. et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR 2023. arXiv:2203.11171 — uso: fuente primaria del mecanismo estudiado
- Yao, S. et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. arXiv:2305.10601 — uso: fuente primaria del mecanismo estudiado
- Snell, C. et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. arXiv:2408.03314 — uso: fuente primaria del mecanismo estudiado
- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948 — uso: fuente primaria del mecanismo estudiado
- OpenAI (2024). *Learning to reason with LLMs* (o1). Blog oficial — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P22 · DeepSeek-R1: incentivar la capacidad de razonamiento mediante aprendizaje por refuerzo	2025	El razonamiento se incentiva con refuerzo puro, sin trazas humanas anotadas; y es el primer LLM de pesos abiertos publicado tras revisión por pares.	notebook
P28 · El prompting de cadena de pensamiento provoca razonamiento en modelos de lenguaje grandes	2022	Descomponer en pasos intermedios desbloquea tareas que el mismo modelo fallaba respondiendo de una vez.	notebook
P29 · Árbol de pensamientos: resolución deliberada de problemas con modelos de lenguaje grandes	2023	Devuelve la búsqueda clásica al razonamiento: explorar varias ramas, evaluarlas y poder retroceder.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define razonamiento y cómputo en tiempo de inferencia sin usar una marca o framework como definición.
2. Explica la relación entre reasoning, test-time compute, verification, search.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

176 — Aprendizaje continuo y adaptación

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `frontier` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **aprendizaje continuo y adaptación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aprendizaje continuo y adaptación usando los conceptos `continual learning`, `adaptation`, `forgetting`, `memory`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`continual learning`, `adaptation`, `forgetting`, `memory`

🗺️ Ubicación en el mapa de la IA

El aprendizaje continuo ataca una limitación estructural del deep learning clásico: las redes se entrenan una vez sobre un dataset fijo y, si luego se ajustan a una tarea nueva, tienden a destruir lo aprendido antes (*catastrophic forgetting*). Hereda directamente de los fundamentos de optimización por gradiente (partes 3-4) y del fine-tuning de modelos (parte 7), y habilita la visión de agentes que operan durante meses acumulando experiencia sin reentrenar desde cero, un requisito de los sistemas evolutivos que integra el capstone de esta parte.

📖 Fundamentos

🧠 El problema: catastrophic forgetting

Una red entrenada por descenso de gradiente ajusta **todos** sus pesos para minimizar la pérdida de la tarea actual. Si tras dominar la tarea A se entrena en la tarea B con el mismo procedimiento, los gradientes de B mueven los pesos sin ninguna restricción que proteja lo que era importante para A. El resultado empírico, documentado desde McCloskey y Cohen (1989), es que el rendimiento en A puede colapsar a niveles de azar

tras pocas épocas de B. No es un fallo de capacidad —la red podría representar ambas tareas— sino de **interferencia**: ambas tareas compiten por los mismos parámetros.

Definiciones precisas:

- **Aprendizaje continuo (continual/lifelong learning)**: entrenar un modelo sobre una secuencia de tareas `T1, T2, …, Tn` viendo los datos de cada tarea solo durante su fase, y evaluando al final sobre *todas* las tareas.
- **Olvido (forgetting)**: caída de rendimiento en `Ti` medida después de entrenar en `Tj` con `j > i`, respecto al rendimiento justo al terminar `Ti`.
- **Transferencia hacia atrás/adelante (backward/forward transfer)**: efecto (positivo o negativo) de aprender una tarea nueva sobre tareas anteriores/posteriores.
- **Estabilidad vs plasticidad**: el dilema central. Máxima estabilidad = congelar pesos (no olvida, no aprende); máxima plasticidad = fine-tuning ingenuo (aprende, olvida). Todo método de aprendizaje continuo es un punto en ese espectro.

🧊 Familia 1: regularización — Elastic Weight Consolidation (EWC)

EWC (Kirkpatrick et al., 2017, arXiv:1612.00796) formaliza una intuición bayesiana: tras aprender A, la posterior sobre los pesos indica cuáles son importantes. Al entrenar B, se penaliza mover los pesos importantes para A:

```
L(θ) = L_B(θ) + (λ/2) · Σ_i F_i · (θ_i − θ*_{A,i})²

θ*_A : pesos óptimos al terminar la tarea A
F_i  : información de Fisher diagonal ≈ E[(∂ log p(y|x,θ)/∂θ_i)²]
       (qué tan sensible es la predicción al peso i → su "importancia")
λ    : cuánto pesa la memoria de A frente al aprendizaje de B
```

Un peso con `F_i` alto queda anclado (resorte rígido); uno con `F_i ≈ 0` queda libre para adaptarse a B. Con `λ → 0` se recupera el fine-tuning ingenuo; con `λ → ∞` el modelo se congela.

🔁 Familia 2: repetición (replay)

En lugar de restringir los pesos, se restringen los **datos**: se guarda un buffer pequeño de ejemplos de tareas anteriores (o un modelo generativo que los sintetiza) y cada minibatch de la tarea nueva se mezcla con ejemplos antiguos. GEM (Lopez-Paz y Ranzato, 2017) además proyecta el gradiente para que no aumente la pérdida sobre el buffer. El replay suele ser el método más robusto en la práctica, al costo de memoria y de posibles problemas de privacidad (retener datos crudos).

🏗️ Familia 3: arquitectura

Asignar capacidad nueva por tarea: Progressive Networks añaden columnas congelando las anteriores; los adaptadores/LoRA por tarea entrenan módulos pequeños dejando el backbone intacto. Eliminan el olvido por construcción, pero el modelo crece con cada tarea y exige saber qué tarea se está resolviendo en inferencia.

📏 Cómo se mide

Con la matriz `R ∈ ℝ^{n×n}` donde `R[i][j]` = rendimiento en la tarea `j` tras entrenar hasta la tarea `i`: precisión media final = `mean_j R[n][j]`; olvido de `j` = `max_i R[i][j] − R[n][j]`; BWT (backward transfer)

= media de `R[n][j] - R[j][j]`.

🧮 Ejemplo trabajado

Modelo de juguete con dos pesos. Tras la tarea A: `θ*_A = (2.0, -1.0)` con Fisher diagonal `F = (5.0, 0.1)` — el peso 1 es crítico para A, el peso 2 casi no importa. La tarea B, por sí sola, preferiría `θ_B = (0.0, 3.0)`, con `L_B(θ) = (θ₁ - 0)² + (θ₂ - 3)²`. Tomamos `λ = 2`.

```
L(θ) = (θ₁)² + (θ₂ - 3)² + (2/2)·[5.0·(θ₁ - 2)² + 0.1·(θ₂ + 1)²]

∂L/∂θ₁ = 2θ₁ + 10(θ₁ - 2)      = 0  →  12θ₁ = 20  →  θ₁ = 1.67
∂L/∂θ₂ = 2(θ₂ - 3) + 0.2(θ₂+1) = 0  →  2.2θ₂ = 5.8 →  θ₂ = 2.64
```

Lectura: el peso 1 (importante para A, `F=5`) se queda cerca de A (1.67 frente al 0.0 que pediría B); el peso 2 (irrelevante para A, `F=0.1`) se mueve casi hasta lo que pide B (2.64 frente a 3.0). Con fine-tuning ingenuo (`λ=0`) ambos irían a (0, 3) y A quedaría destruida. Ese es todo el mecanismo de EWC: resortes con rigidez proporcional a la importancia.

📊 Propiedades y comparación

Método	Memoria extra	¿Crece el modelo?	¿Necesita datos viejos?	Olvido	Límite principal
Fine-tuning ingenuo	Ninguna	No	No	Severo	Destruye tareas previas
EWC (regularización)	O(2·params) (θ*, F)	No	No	Moderado	F diagonal es aproximación; se degrada con muchas tareas
Replay con buffer	Buffer de ejemplos	No	Sí (muestra)	Bajo	Privacidad y tamaño del buffer
GEM	Buffer + proyección	No	Sí	Bajo	Costo de resolver la proyección por paso
Progressive / LoRA por tarea	Módulos por tarea	Sí	No	Nulo por construcción	Crecimiento lineal; requiere identidad de tarea

⚠️ Errores conceptuales frecuentes

1. **"El olvido se arregla con más épocas o más datos de la tarea nueva."** Al contrario: más optimización sobre B sin protección aumenta la interferencia sobre A.
2. **"EWC guarda los datos de la tarea anterior."** No: guarda solo `θ*_A` y `F` (estadísticos de los pesos). Eso es precisamente su ventaja de privacidad frente al replay, y también su debilidad (la aproximación diagonal pierde correlaciones).
3. **"Evaluar solo la última tarea basta."** La métrica del aprendizaje continuo es la matriz completa: un método puede ganar en `Tn` habiendo arrasado `T1..Tn-1`.
4. **"Los LLM con RAG ya hacen aprendizaje continuo."** Añadir contexto recuperado no modifica los pesos: es memoria externa, no consolidación. Resuelve otro problema (conocimiento actualizado) sin tocar el dilema estabilidad-plasticidad.
5. **"Congelar el modelo elimina el problema."** Lo elimina junto con la adaptación: un sistema congelado se degrada frente a *distribution shift*, que es la razón por la que se quería aprender continuamente.

🚀 Del aprendizaje a la operación

El laboratorio ilustra el contrato con un escenario determinista; un sistema real necesita: detección de *drift* que dispare la adaptación (no reentrenar por calendario ciego), un conjunto de regresión congelado por cada capacidad antigua que actúe como prueba de no-olvido antes de promover pesos nuevos, política explícita de retención de datos si se usa replay (privacidad, GDPR), y un plan de rollback: en producción, el peor olvido no es el del modelo sino el del equipo que no versionó el checkpoint anterior.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("frontier")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kirkpatrick, J. et al. (2017). *Overcoming catastrophic forgetting in neural networks*. PNAS 114(13). DOI 10.1073/pnas.1611835114 · arXiv:1612.00796 — uso: fuente primaria del mecanismo estudiado
- Parisi, G. I. et al. (2019). *Continual Lifelong Learning with Neural Networks: A Review*. Neural Networks 113. arXiv:1802.07569 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I. J. et al. (2013). *An Empirical Investigation of Catastrophic Forgetting in Gradient-Based Neural Networks*. arXiv:1312.6211 — uso: fuente primaria del mecanismo estudiado
- Lopez-Paz, D. y Ranzato, M. (2017). *Gradient Episodic Memory for Continual Learning*. NeurIPS 2017. arXiv:1706.08840 — uso: fuente primaria del mecanismo estudiado
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*, cap. 8 (optimización). deeplearningbook.org — uso: desarrollo extendido del tema

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P142 · Interferencia catastrófica en redes conexionistas: el problema del aprendizaje secuencial	1989	Documenta que aprender una tarea nueva borra la anterior de golpe, y que no es una degradación gradual sino un colapso.	notebook
P145 · Superar el olvido catastrófico en redes neuronales	2017	Frena selectivamente los pesos que importaban para las tareas anteriores y deja libres los demás, con una penalización derivada de la información de Fisher.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 8 · obra de referencia de la parte 14
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA

📝 Evaluación completa

❓ Preguntas

1. Define aprendizaje continuo y adaptación sin usar una marca o framework como definición.
2. Explica la relación entre continual learning, adaptation, forgetting, memory.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

177 — Privacidad diferencial y aprendizaje federado

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `safety` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **privacidad diferencial y aprendizaje federado** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar privacidad diferencial y aprendizaje federado usando los conceptos `differential privacy`, `federated`, `secure aggregation`, `leakage`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`differential privacy`, `federated`, `secure aggregation`, `leakage`

🗺️ Ubicación en el mapa de la IA

La privacidad diferencial y el aprendizaje federado responden a la misma pregunta desde extremos opuestos: ¿cómo aprender de datos que no deberían exponerse? La primera da una **garantía matemática** sobre lo que la salida revela de un individuo; el segundo es una **arquitectura** donde los datos nunca abandonan el dispositivo. Heredan de la estadística y la optimización distribuida, y son condición de entrada para IA en salud, banca y móviles (Gboard entrena federado desde 2017). Sin estas técnicas, gran parte de los datos del mundo queda legalmente fuera del alcance del aprendizaje automático.

📖 Fundamentos

🔐 Privacidad diferencial (ε-DP)

Definición (Dwork, 2006): un mecanismo aleatorizado `M` satisface **ε-privacidad diferencial** si para todo par de datasets vecinos `D, D'` (difieren en un individuo) y todo conjunto de salidas `S`:

```
Pr[M(D) ∈ S] ≤ e^ε · Pr[M(D') ∈ S]
```

Lectura: la presencia o ausencia de *cualquier* persona cambia la distribución de la salida a lo sumo en un factor `e^ε`. Con `ε = 0.1`, `e^ε ≈ 1.105`: un observador casi no puede distinguir si estás en los datos. La garantía es del **mecanismo**, no de los datos: vale contra cualquier atacante, con cualquier conocimiento auxiliar, hoy y en el futuro. Eso la diferencia de la anonimización clásica (k-anonimato), rota una y otra vez por ataques de re-identificación con datos auxiliares (caso Netflix Prize).

🎲 Mecanismo de Laplace

Para una consulta numérica `f(D)` se define la **sensibilidad global** `Δf = max_{D,D'} |f(D) − f(D')|` (cuánto puede cambiar la respuesta por culpa de una sola persona). El mecanismo publica:

```
M(D) = f(D) + Lap(b)     con  b = Δf / ε
```

donde `Lap(b)` es ruido Laplace con densidad `(1/2b)·exp(−|x|/b)`. Ejemplo: un conteo ("¿cuántos pacientes tienen diabetes?") tiene `Δf = 1`; con `ε = 0.5` se suma ruido de escala `b = 2`. Propiedades clave:

- **Composición**: responder `k` consultas con `ε` cada una consume `k·ε` en total (el "presupuesto de privacidad" se gasta; no es renovable).
- **Post-procesamiento**: cualquier función de una salida ε-DP sigue siendo ε-DP.
- **(ε, δ)-DP**: relajación usada en deep learning (DP-SGD, Abadi et al. 2016): recorte de gradientes por ejemplo + ruido gaussiano + contabilidad del presupuesto.

📱 Aprendizaje federado — FedAvg

FedAvg (McMahan et al., 2017, arXiv:1602.05629) entrena un modelo global sin centralizar datos:

```
Servidor: inicializa w_0
Por ronda t = 1..T:
  1. Selecciona una fracción C de los K clientes
  2. Envía w_t a cada cliente seleccionado
  3. Cada cliente k entrena E épocas locales sobre sus n_k ejemplos → w_t^k
  4. Servidor agrega:  w_{t+1} = Σ_k (n_k / n) · w_t^k    (media ponderada)
```

Desafíos propios: datos **no-IID** (cada cliente tiene una distribución distinta, lo que sesga y desestabiliza la media), clientes intermitentes, y costo de comunicación (por eso `E > 1` épocas locales: menos rondas, más cómputo local).

🕳️ Federado NO implica privado

Los gradientes filtran información: los ataques de **inversión de gradientes** (Zhu et al., 2019, "Deep Leakage from Gradients") reconstruyen imágenes y textos de entrenamiento a partir de las actualizaciones. Por eso el sistema completo combina: federado (los datos no viajan) + **agregación segura** (el servidor solo ve la suma cifrada de actualizaciones, Bonawitz et al. 2017) + **DP** (ruido sobre la actualización agregada, garantía formal). Cada pieza cubre un ataque distinto.

🧱 Lo que estas técnicas no protegen: datos en uso

Privacidad diferencial limita cuánto revela un mecanismo sobre la contribución de una persona; aprendizaje federado evita centralizar datos crudos. Ninguno impide por sí mismo que el host lea memoria durante una

inferencia o agregación. Secure aggregation protege la suma distribuida, pero no equivale a aislar todo el runtime.

La clase 183 añade el control que falta en esa dimensión: TEE, medición y remote attestation antes de liberar claves. La arquitectura completa combina técnicas según amenaza; no las presenta como sinónimos intercambiables de «privacidad».

🧮 Ejemplo trabajado

Un hospital publica el conteo de pacientes con cierta condición: `f(D) = 42`. Consulta de conteo → `Δf = 1`. Presupuesto elegido: `ε = 0.5`.

```
Escala del ruido:  b = Δf / ε = 1 / 0.5 = 2
Mecanismo:         M(D) = 42 + Lap(2)

Garantía sobre un individuo (D' sin la persona X, f(D') = 41):
para cualquier salida s, la densidad cumple
p_D(s) / p_D'(s) = exp((|s−41| − |s−42|)/2) ≤ exp(1/2) = e^0.5 ≈ 1.65

Ruido esperado: E|Lap(2)| = b = 2  →  error típico de ±2 sobre 42 (≈5 %)
Tres consultas iguales con ε=0.5 cada una → presupuesto total gastado: ε=1.5
  (promediar las tres respuestas reduce el ruido... y por eso mismo triplica ε)
```

La tensión es visible con números: más precisión (b pequeño) exige más ε (menos privacidad), y repetir consultas gasta presupuesto aunque "parezcan inocentes".

📊 Propiedades y comparación

Enfoque	¿Datos salen del origen?	Garantía formal	Contra qué protege	Costo principal
Anonimización clásica (k-anon)	Sí (transformados)	No	Ataques ingenuos	Re-identificación con datos auxiliares
DP central	Sí (al curador confiable)	ε-DP en la salida	Inferencia sobre individuos	Ruido ↔ utilidad; requiere confiar en el curador
DP local	No (se perturba en el dispositivo)	ε-DP por envío	Curador deshonesto	Mucho más ruido para igual utilidad
Federado (FedAvg solo)	No (viajan gradientes)	No	Centralización de datos crudos	Fuga por inversión de gradientes
Federado + SecAgg + DP	No	(ε,δ)-DP	Servidor curioso e inferencia	Comunicación, cripto, ruido

⚠️ Errores conceptuales frecuentes

1. **"Federado ya es privado."** No: los gradientes filtran datos (deep leakage). La privacidad formal la aportan SecAgg + DP encima de la arquitectura federada.
2. **"ε-DP protege el secreto del dataset entero."** Protege la contribución de *individuos*; estadísticas poblacionales (que fumar causa cáncer) se aprenden igual — ese es justamente el objetivo.
3. **"Anonimizar quitando nombres equivale a DP."** El caso Netflix/IMDb demostró que los datos auxiliares re-identifican; DP es una propiedad del mecanismo que resiste cualquier dato auxiliar, presente o futuro.
4. **"El presupuesto ε se renueva por consulta."** Se **compone**: k consultas de ε gastan k·ε sobre los mismos datos. Ignorarlo anula la garantía en la práctica.
5. **"Más ruido siempre es más seguro."** Sin calibrar a Δf y sin contabilidad, el ruido puede ser a la vez insuficiente para la garantía y excesivo para la utilidad; la calibración `b = Δf/ε` es exacta, no heurística.

🚀 Del aprendizaje a la operación

Entre este núcleo y un despliegue real faltan: la **contabilidad de presupuesto** sobre todo el ciclo de vida (cada release del modelo gasta ε; Apple y el US Census publican los suyos), infraestructura federada tolerante a clientes que se caen a mitad de ronda, evaluación de utilidad por subgrupos (el ruido DP degrada más a las minorías del dataset), y revisión legal: DP con ε alto puede ser teatro de privacidad — el número exacto de ε importa y debe publicarse.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("safety")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;

- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Dwork, C., McSherry, F., Nissim, K. y Smith, A. (2006). *Calibrating Noise to Sensitivity in Private Data Analysis*. TCC 2006. DOI 10.1007/11681878_14 — uso: fuente primaria del mecanismo estudiado
- Dwork, C. y Roth, A. (2014). *The Algorithmic Foundations of Differential Privacy*. PDF oficial — uso: referencia consultada en su fuente original
- McMahan, H. B. et al. (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data* (FedAvg). AISTATS 2017. arXiv:1602.05629 — uso: fuente primaria del mecanismo estudiado
- Abadi, M. et al. (2016). *Deep Learning with Differential Privacy* (DP-SGD). CCS 2016. arXiv:1607.00133 — uso: fuente primaria del mecanismo estudiado
- Bonawitz, K. et al. (2017). *Practical Secure Aggregation for Privacy-Preserving Machine Learning*. CCS 2017. DOI 10.1145/3133956.3133982 — uso: fuente primaria del mecanismo estudiado
- Zhu, L., Liu, Z. y Han, S. (2019). *Deep Leakage from Gradients*. NeurIPS 2019. arXiv:1906.08935 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P143 · Calibrar el ruido a la sensibilidad en el análisis privado de datos	2006	Da una definición formal de privacidad que no depende de qué sepa el atacante, y un mecanismo concreto para cumplirla.	notebook
P146 · Aprendizaje eficiente en comunicación de redes profundas con datos descentralizados	2017	Entrena un modelo compartido sin que los datos salgan del dispositivo, promediando modelos en vez de recoger registros.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define privacidad diferencial y aprendizaje federado sin usar una marca o framework como definición.
2. Explica la relación entre differential privacy, federated, secure aggregation, leakage.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

178 — IA para programación y modernización

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `agent` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ia para programación y modernización** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ia para programación y modernización usando los conceptos `coding agents`, `tests`, `migration`, `legacy`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`coding agents`, `tests`, `migration`, `legacy`

🗺️ Ubicación en el mapa de la IA

La programación es el dominio donde los LLM encontraron antes su encaje económico: el código tiene un verificador natural (compilar, ejecutar tests) que convierte la generación probabilística en un bucle con señal de corrección. Hereda de los LLM y el fine-tuning (partes 6-7) y de la agentica con herramientas (parte 9), y es el banco de pruebas de la computación en tiempo de inferencia (clase 175): muestrear muchas soluciones y filtrar por tests es test-time compute aplicado. La modernización de sistemas legados es su frontera económicamente más pesada.

📖 Fundamentos

🤖 De autocompletar a agente

Tres generaciones con contratos distintos:

1. **Autocompletado (Codex, 2021, arXiv:2107.03374)**: dado un prefijo (docstring, firma), el modelo genera la continuación. Evaluación: funciones aisladas (HumanEval: 164 problemas con tests unitarios ocultos).

2. **Chat/instrucción**: el modelo dialoga sobre código, explica y refactoriza; el humano integra. La evaluación se vuelve difusa (no hay harness automático).
3. **Agente de código (SWE-bench, 2023, arXiv:2310.06770)**: dado un *issue* real de GitHub y el repositorio completo, el agente debe localizar los archivos, editar, ejecutar tests y producir un parche. Se acepta si pasan los tests de la corrección real (`FAIL_TO_PASS`) sin romper los existentes (`PASS_TO_PASS`).

El salto de 1→3 cambia la dificultad: HumanEval mide síntesis local de ~10 líneas; SWE-bench exige navegación de repos de cientos de miles de líneas, comprensión de convenciones y edición multiarchivo. Los primeros sistemas puntuaban <2 % en SWE-bench cuando ya superaban 90 % en HumanEval.

📐 La métrica pass@k

Con generación estocástica, "¿funciona?" es una probabilidad. **pass@k** = P(al menos 1 de k muestras pasa todos los tests). El estimador insesgado (Chen et al., 2021): se generan `n ≥ k` muestras, `c` de ellas correctas, y

```
pass@k = 1 - C(n-c, k) / C(n, k)
```

(elegir k muestras y que ninguna sea correcta, complementado). Calcular ingenuamente `1 - (1 - c/n)^k` sesga el resultado; la fórmula combinatoria es exacta.

🔁 El bucle del agente de código

```
repetir hasta presupuesto:
  1. LOCALIZAR   buscar archivos/símbolos relevantes al issue (grep, mapa del repo)
  2. PROPONER    editar código (parche mínimo, no reescritura)
  3. VERIFICAR   compilar + ejecutar tests (los previos y los del issue)
  4. LEER SEÑAL  traceback/diff → siguiente iteración
aceptar solo si: tests nuevos pasan Y tests previos siguen pasando
```

La calidad del sistema depende menos del modelo que del **harness**: qué contexto se recupera, qué tests existen, cómo se limita el radio de la edición. Sin tests, el bucle degenera en "parece correcto", que es precisamente lo que no se puede auditar. La industria llamó a esta capa **harness engineering** (OpenAI, 2026); la ecuación *Agente = Modelo + Harness* que introduce la clase 113 tiene aquí su caso de uso más maduro.

📋 Spec-driven development: la especificación como contrato

Cuando el agente de código es capaz de ejecutar el bucle completo, el cuello de botella se desplaza a la *entrada*: ¿qué le pedimos exactamente? **Spec-driven development (SDD)** responde convirtiendo la especificación en el artefacto principal del trabajo:

```
1. SPEC      qué debe hacer el cambio, con criterios de aceptación VERIFICABLES
             (idealmente: tests que hoy fallan y deberán pasar)
2. PLAN      el agente propone descomposición y archivos afectados; el humano revisa
             ANTES de que exista código
3. IMPLEMENT el agente ejecuta el plan unidad por unidad, verificando cada una
4. VERIFY    la spec se re-evalúa completa: criterios de aceptación + regresiones
```

Tres observaciones para no comprar el término sin crítica. Primera: SDD es un subconjunto de context engineering — la spec es el contexto de más alta señal que existe, porque define éxito de forma verificable (compárese con `FAIL_TO_PASS` de SWE-bench: un benchmark de agentes *es* una spec ejecutable). Segunda: la objeción "¿no es waterfall con IA?" es parcialmente justa; SDD funciona cuando la spec puede escribirse antes (correcciones, migraciones, features acotadas) y estorba en exploración genuina, donde el bucle iterativo de la sección anterior es el flujo correcto. Tercera: la spec desactualizada es deuda nueva — si el código diverge de la spec y nadie la corrige, el próximo agente implementará contra un contrato falso. La modernización de legado (siguiente sección) es SDD en su forma extrema: los tests de caracterización *son* la spec, extraída del comportamiento real.

🏚️ Modernización de legado

El caso económico dominante: migrar COBOL/Java 6/Python 2 a plataformas mantenibles. El principio (Feathers): *legacy code = código sin tests*. El flujo asistido por IA:

1. **Tests de caracterización**: generar tests que fijan el comportamiento ACTUAL (incluidos bugs), no el deseado — son la red antes de tocar nada.
2. **Traducción por unidades pequeñas** con equivalencia verificada contra esos tests.
3. **Revisión humana** de todo lo que toque I/O, dinero, fechas o concurrencia, donde la equivalencia semántica entre lenguajes es más traicionera.

La IA acelera los pasos 1-2; el riesgo es traducir "lo que parece que hace" en lugar de "lo que hace": una migración sin tests de caracterización no es modernización, es reescritura con esperanza.

🔭 Actualización 2026: del agente de código al agente investigador

El siguiente salto no es un autocompletado mejor, sino **delegar ciclos de trabajo largos y verificables**. OpenAI reportó agentes que, bajo dirección humana, ejecutan tareas de investigación acotadas que tomarían días a una persona: navegan código, modifican experimentos, los ejecutan y comparan resultados. Anthropic publicó un caso complementario en alineación: el agente buscó literatura, propuso datos y métodos, entrenó modelos y evaluó diez clases de fallo, con pruebas retenidas y un agente monitor.

Esto amplía el bucle de esta clase:

```
issue → parche → tests
pregunta → hipótesis → experimento → resultado → crítica → siguiente experimento
```

La continuidad es el **verificador**. Los tests se convierten en benchmarks, protocolos experimentales y criterios de parada; el repositorio se convierte en cuaderno de laboratorio. Lo que todavía no cambia es la responsabilidad: las fuentes son autoestudios de los propios laboratorios, las tareas estaban acotadas y las personas siguieron fijando prioridades y juzgando qué resultados perseguir.

🧮 Ejemplo trabajado

Un modelo genera `n = 5` soluciones para un problema; `c = 2` pasan los tests.

```
pass@1 = 1 − C(3,1)/C(5,1) = 1 − 3/5 = 0.40
pass@3 = 1 − C(3,3)/C(5,3) = 1 − 1/10 = 0.90

comprobación ingenua para k=3: 1 − (1 − 0.4)³ = 1 − 0.216 = 0.784  ≠ 0.90
```

La fórmula ingenua subestima porque muestrea "con reemplazo" soluciones que en realidad son un conjunto fijo. Lectura práctica: con 40 % de aciertos por muestra, basta muestrear 3 veces y filtrar por tests para llegar al 90 % — **si y solo si** existen tests que hagan de filtro. Ese "si" es todo el asunto.

📊 Propiedades y comparación

Nivel	Benchmark típico	Unidad de trabajo	Verificación	Falla característica
Autocompletado	HumanEval (pass@k)	Función aislada	Tests unitarios ocultos	Memorizar el benchmark
Chat de código	Evaluación humana	Fragmento/explicación	Juicio del programador	Plausible pero incorrecto
Agente de repo	SWE-bench (% resuelto)	Issue real + repo	FAIL_TO_PASS + PASS_TO_PASS	Parche que sobreajusta al test
Migración legado	Suites de caracterización	Módulo/sistema	Equivalencia conductual	Traducir la intención, no el comportamiento

⚠️ Errores conceptuales frecuentes

1. **"90 % en HumanEval ≈ 90 % de un programador."** HumanEval mide funciones aisladas de decenas de líneas; los mismos modelos puntuaban <2 % en SWE-bench. El benchmark define qué se midió; extrapolar entre niveles no es válido.
2. **"pass@k se calcula como 1−(1−c/n)^k."** El estimador correcto es el combinatorio `1 − C(n−c,k)/C(n,k)`; el ingenuo está sesgado (ver ejemplo).
3. **"Si los tests pasan, el parche es correcto."** Los tests son una muestra del contrato: un agente puede sobreajustar al test (hardcodear el caso) o romper comportamiento no cubierto. De ahí PASS_TO_PASS y la revisión humana.
4. **"La IA puede migrar el sistema legado leyendo el código."** Sin tests de caracterización no hay definición ejecutable de 'equivalente'; la traducción plausible es el modo de fallo más caro de la modernización.

5. **"La contaminación no importa si el benchmark es público."** Al contrario: repos y soluciones públicos entran al pretraining; por eso existen variantes verificadas y con corte temporal (p. ej. SWE-bench Verified) y hay que mirar la fecha de los issues frente al corte del modelo.

🚀 Del aprendizaje a la operación

Para operar un asistente/agente de código real faltan: sandbox de ejecución aislado (el agente ejecuta código arbitrario), política de secretos (el repo contiene credenciales que no deben llegar al contexto), telemetría de tasa de aceptación y de regresiones post-merge (la métrica que importa no es pass@k sino "parches revertidos por 100 fusionados"), y límites de radio: un agente que puede editar CI/CD o dependencias necesita revisión obligatoria — el bucle editar-testear no cubre ataques de cadena de suministro.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("agent")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Chen, M. et al. (2021). *Evaluating Large Language Models Trained on Code* (Codex, HumanEval, pass@k). arXiv:2107.03374 — uso: fuente primaria del mecanismo estudiado
- Jimenez, C. E. et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770 · swebench.com — uso: fuente primaria del mecanismo estudiado
- Li, Y. et al. (2022). *Competition-level code generation with AlphaCode*. Science 378(6624). DOI 10.1126/science.abq1158 — uso: fuente primaria del mecanismo estudiado
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall. Ficha editorial — uso: referencia consultada en su fuente original
- Austin, J. et al. (2021). *Program Synthesis with Large Language Models* (MBPP). arXiv:2108.07732 — uso: fuente primaria del mecanismo estudiado
- OpenAI (2026). *Harness engineering: leveraging Codex in an agent-first world*. openai.com/index/harness-engineering — uso: referencia consultada en su fuente original
- OpenAI (2026). *Research acceleration: The view inside OpenAI*. openai.com/index/research-acceleration-view-inside-openai — uso: evidencia primaria reciente, tratada como frontera y no como validación independiente
- Anthropic (2026). *Automated researchers can reliably mitigate alignment failures*. anthropic.com/research/automated-researchers-mitigate-alignment-failures — uso: segundo caso de ciclo experimental agéntico con límites declarados
- GitHub — *Spec Kit* (toolkit de spec-driven development para agentes de código). github.com/github/spec-kit — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P51 · SWE-bench: ¿pueden los modelos resolver incidencias reales de GitHub?	2023	Cambia el criterio de evaluación: no si el código parece bien, sino si los tests del repositorio real pasan.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define ia para programación y modernización sin usar una marca o framework como definición.
2. Explica la relación entre coding agents, tests, migration, legacy.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

179 — IA para ciberseguridad y defensa

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `safety` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ia para ciberseguridad y defensa** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ia para ciberseguridad y defensa usando los conceptos `cybersecurity`, `detection`, `response`, `dual-use`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`cybersecurity`, `detection`, `response`, `dual-use`

🗺️ Ubicación en el mapa de la IA

La ciberseguridad es un dominio adversarial por definición: a diferencia de la visión o el clima, aquí el fenómeno a modelar **responde** a las defensas y las evade. La IA defensiva hereda de la clasificación y detección de anomalías (partes 3-4) y de los agentes con herramientas (parte 9), y a la vez alimenta la seguridad de la propia IA (clases de la parte 13): los mismos LLM que ayudan a triar alertas son superficie de ataque por inyección de prompts. Es también el caso de estudio canónico del problema de doble uso.

📖 Fundamentos

🛡️ Dónde encaja la IA en la defensa

Cuatro funciones con madurez muy distinta:

- **Detección**: clasificar eventos (tráfico, procesos, correos) como benignos o maliciosos. Por firma (patrones conocidos, precisión alta, ciego a lo nuevo) o por anomalía (desviación de un perfil normal, cubre lo nuevo, inunda de falsos positivos).

- **Triaje**: priorizar las miles de alertas diarias de un SOC (centro de operaciones de seguridad); es hoy el uso más efectivo de los LLM — resumir, correlacionar, enriquecer contexto — porque el humano permanece en la decisión.
- **Respuesta**: contener (aislar host, revocar credenciales). La automatización aquí es delicada: una respuesta automática equivocada es un autoataque de denegación de servicio.
- **Inteligencia**: mapear observaciones a tácticas y técnicas conocidas (el marco MITRE ATT&CK es la taxonomía estándar de comportamientos adversarios).

📉 La falacia de la tasa base (el resultado central)

El resultado más importante para detección es de Axelsson (1999): en detección de intrusiones, la utilidad la domina la **prevalencia**, no la precisión del modelo. Con eventos maliciosos rarísimos, incluso un detector excelente produce casi solo falsas alarmas:

```
P(intrusión | alerta) =  TPR·p / (TPR·p + FPR·(1−p))      (Bayes)

TPR: tasa de detección    FPR: tasa de falsa alarma    p: prevalencia
```

Si $p \approx 10^{-5}..10^{-4}$ (realista en tráfico de red), el término $FPR\cdot(1-p)$ aplasta al numerador salvo que FPR sea astronómicamente bajo. Conclusión operativa: **reducir FPR vale más que aumentar TPR**, y la métrica que importa es la precisión de la cola de alertas que un analista puede revisar, no el AUC global.

⚔️ Asimetría ataque/defensa

- El atacante necesita **una** vía; el defensor debe cubrir **todas**.
- El atacante puede probar contra la defensa hasta evadirla (los clasificadores de malware sufren evasión adversarial dirigida); el defensor no elige la distribución de los datos, que además es no estacionaria (*concept drift*: el malware de este año no se parece al del dataset de entrenamiento).
- Sommer y Paxson (2010) explican por qué el ML "que funciona en el paper" fracasa en detección operativa: la anomalía no implica malicia, los costos de error son asimétricos y los datasets de laboratorio no representan redes reales.

La IA también es **dual**: los mismos modelos generan phishing convincente, buscan vulnerabilidades y automatizan reconocimiento. La política defensiva no puede asumir un adversario sin IA.

🧭 LLM en el SOC: contrato seguro

```
Entrada:  alerta + contexto (logs, activo, historial) — SANITIZADOS
Tarea:    resumir, correlacionar con ATT&CK, proponer hipótesis y pasos
Salida:   recomendación con evidencia citada, NUNCA acción directa
Guardia:  el contenido del atacante (correos, payloads) es DATOS, no
          instrucciones → riesgo de prompt injection si se pega crudo al LLM
```

🧮 Ejemplo trabajado

Un IDS analiza 1,000,000 de eventos/día; 100 son realmente maliciosos (prevalencia $p = 10^{-4}$). El detector tiene TPR = 0.99 y FPR = 0.01 ("99 % de precisión" en el folleto).

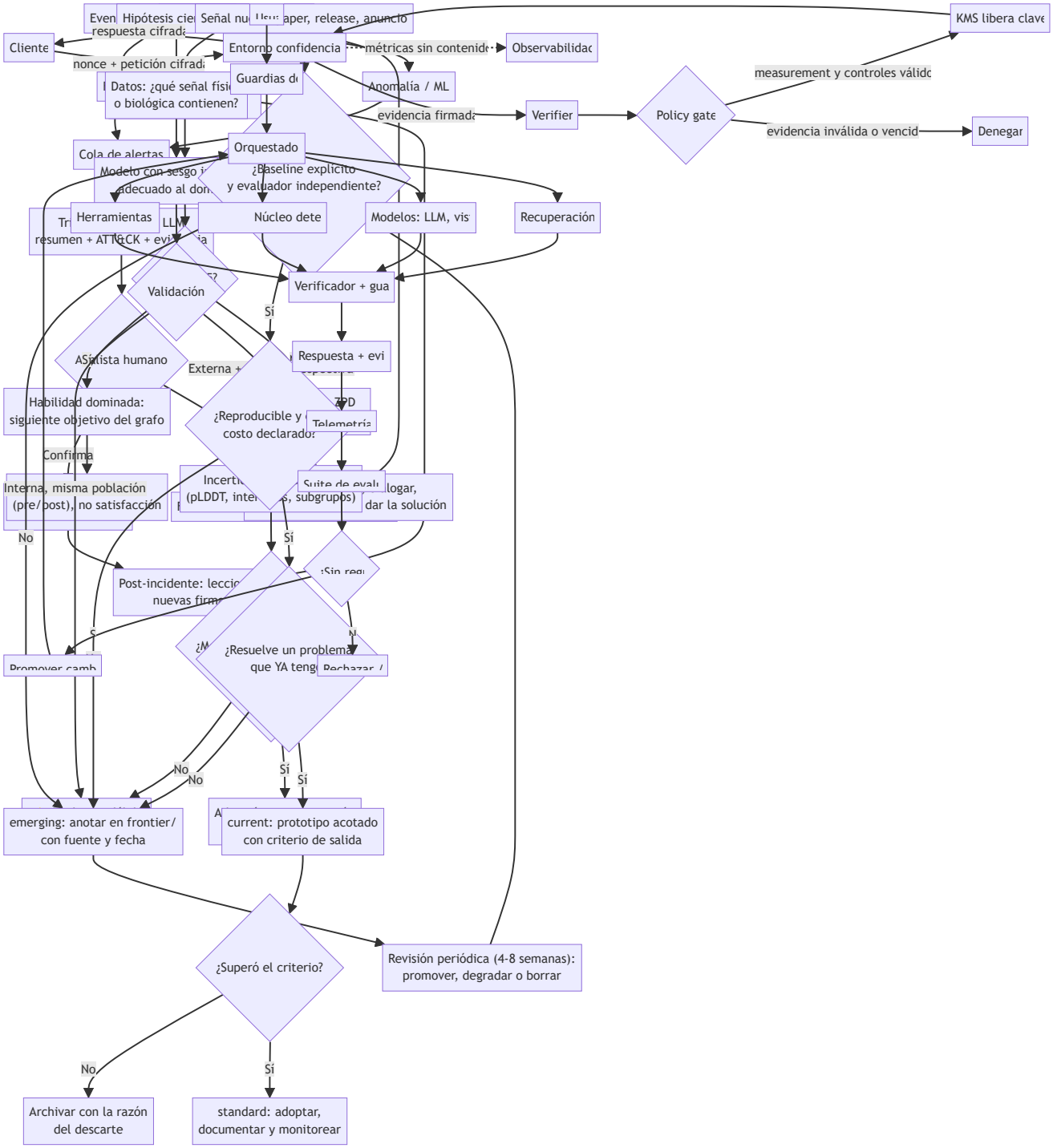
```
Verdaderos positivos: 0.99 × 100        =      99 alertas correctas
Falsos positivos:     0.01 × 999,900    =   9,999 falsas alarmas

P(malicioso | alerta) = 99 / (99 + 9,999) = 99/10,098 ≈ 0.0098  →  ≈ 1 %
```

El "detector del 99 %" produce 10,098 alertas diarias de las cuales el 99 % son falsas. Un equipo que revisa 200 alertas/día encontraría ~2 incidentes reales. Repetir el cálculo con FPR = 0.001 da 99/(99+1,000) ≈ 9 %: **bajar FPR ×10 ayuda ~×9; subir TPR a 0.999 ayuda un 1 %**. Esa es la falacia de la tasa base en números.

📊 Propiedades y comparación

Enfoque	Detecta lo nuevo	FPR típico	Explicabilidad	Evasión por el atacante
Firmas/reglas	No	Muy bajo	Alta (regla exacta)	Trivial (mutar el patrón)
Anomalía estadística	Sí (desviaciones)	Alto	Media	Envenenar el perfil "normal" lentamente
Clasificador ML supervisado	Parcial (generaliza)	Medio	Baja	Ejemplos adversariales, drift
LLM de triaje	n/a (no detecta: prioriza)	n/a	Alta (narrativa)	Prompt injection en el contenido



⚠️ Errores conceptuales frecuentes

1. **"99 % de exactitud = detector utilizable."** Con prevalencia 10^{-4}, ese detector entrega ~1 % de precisión por alerta (ejemplo trabajado). La exactitud global es la métrica equivocada en clases raras.
2. **"Anómalo = malicioso."** La mayoría de las anomalías son administradores haciendo cosas raras legítimas; la anomalía es una pista, no un veredicto.
3. **"Automatizar la respuesta ahorra el analista."** Una contención automática con falsos positivos es un ataque de denegación de servicio autoinfligido; la automatización segura empieza por el triaje, no por la respuesta.

4. **"El modelo se entrena una vez."** El adversario se adapta al detector (evasión dirigida) y el tráfico cambia (drift): la detección es un proceso, no un artefacto.
5. **"El LLM puede leer el correo del atacante sin riesgo."** El contenido controlado por el adversario puede contener instrucciones (prompt injection); debe tratarse como datos con sanitización y sin capacidad de acción directa.

🚀 Del aprendizaje a la operación

Un despliegue real añade: medición continua de la precisión de la cola de alertas (no del AUC offline), presupuesto explícito de alertas/día por analista, playbooks donde el LLM propone y el humano ejecuta (separación de privilegios), red team periódico que ataque también al propio pipeline de IA (evasión y prompt injection), y trazabilidad completa: en un incidente legal, "el modelo lo marcó" sin evidencia citable no sostiene ninguna decisión.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("safety")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Axelsson, S. (2000). *The base-rate fallacy and the difficulty of intrusion detection*. ACM TISSEC 3(3). DOI 10.1145/357830.357849 — uso: fuente primaria del mecanismo estudiado
- Sommer, R. y Paxson, V. (2010). *Outside the Closed World: On Using Machine Learning for Network Intrusion Detection*. IEEE S&P 2010. DOI 10.1109/SP.2010.25 — uso: fuente primaria del mecanismo estudiado
- MITRE ATT&CK — base de conocimiento de tácticas y técnicas adversarias. attack.mitre.org — uso: referencia consultada en su fuente original
- NIST Cybersecurity Framework 2.0 (2024). nist.gov/cyberframework — uso: marco normativo de referencia
- Apruzzese, G. et al. (2023). *The Role of Machine Learning in Cybersecurity*. ACM Digital Threats 4(1). DOI 10.1145/3545574 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es

> `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P144 · Fuera del mundo cerrado: sobre el uso de aprendizaje automático para detectar intrusiones	2010	Explica por qué el aprendizaje automático funciona peor en seguridad que en cualquier otro dominio, y por qué la culpa no es del modelo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

Normas y documentación oficial que aplica esta clase: nist.gov/cyberframework

📝 Evaluación completa

❓ Preguntas

1. Define ia para ciberseguridad y defensa sin usar una marca o framework como definición.
2. Explica la relación entre cybersecurity, detection, response, dual-use.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

180 — IA para educación y aprendizaje adaptativo

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `agent` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ia para educación y aprendizaje adaptativo** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ia para educación y aprendizaje adaptativo usando los conceptos `education`, `tutoring`, `assessment`, `pedagogy`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`education`, `tutoring`, `assessment`, `pedagogy`

🗺️ Ubicación en el mapa de la IA

La educación fue uno de los primeros sueños de la IA aplicada (los "sistemas tutores inteligentes" datan de los años 70) y el hallazgo que la motiva sigue vigente: Bloom (1984) midió que la tutoría individual mejora el rendimiento ~2 desviaciones estándar sobre la clase tradicional — el "problema 2 sigma": lograr ese efecto a un costo escalable. Esta clase hereda de la inferencia bayesiana (parte 3) y de los LLM conversacionales (partes 6-7), y conecta con la evaluación de sistemas (parte 13): un tutor que no mide lo que el estudiante sabe no es adaptativo, es un PDF con chat.

📖 Fundamentos

🎓 Los tres problemas del tutor adaptativo

1. **Modelar al estudiante**: estimar qué sabe (knowledge tracing).
2. **Elegir la siguiente actividad**: ni trivial ni imposible (la zona de desarrollo próximo, ZPD, de Vygotsky: lo que el estudiante puede hacer con ayuda pero no solo; ahí ocurre el aprendizaje).

3. **Dar retroalimentación**: pistas graduadas en lugar de la solución (la tutoría eficaz, VanLehn 2011, funciona por andamiaje, no por explicación magistral).

📈 Bayesian Knowledge Tracing (BKT)

El modelo clásico (Corbett y Anderson, 1995) trata cada habilidad como una variable binaria latente L (dominada / no dominada) y actualiza P(L) con cada respuesta. Cuatro parámetros por habilidad:

```
P(L0)  probabilidad inicial de dominio
P(T)   transición: aprender la habilidad tras una oportunidad de práctica
P(S)   slip: fallar aunque se domina        ("resbalón")
P(G)   guess: acertar sin dominar            (adivinar)

Observación correcta:
  P(L | correcto) = P(L)(1-P(S)) / [ P(L)(1-P(S)) + (1-P(L))P(G) ]
Observación incorrecta:
  P(L | error)    = P(L)P(S)     / [ P(L)P(S)     + (1-P(L))(1-P(G)) ]
Después, en ambos casos, oportunidad de aprender:
  P(L') = P(L|obs) + (1 - P(L|obs))·P(T)
```

Es un filtro bayesiano de dos estados (un HMM): actualización por evidencia + deriva por aprendizaje. Con P(L) > umbral (típico 0.95) se declara "dominio" y se avanza.

Deep Knowledge Tracing (Piech et al., 2015) sustituye el modelo por habilidad por una RNN sobre la secuencia completa de interacciones: captura correlaciones entre habilidades y prerequisitos implícitos, a cambio de perder la interpretabilidad de los 4 parámetros (¿por qué el modelo cree que no domino fracciones?) y de necesitar muchos más datos.

🎯 Elegir actividad: ZPD como regla de decisión

Con el modelo de estudiante en la mano, la política clásica es mantener la probabilidad de éxito de la siguiente tarea en una banda intermedia (≈ 0.6-0.8): demasiado fácil no enseña, demasiado difícil frustra y no da señal utilizable. Esto convierte la ZPD en un criterio computable: elegir el ítem cuya dificultad estimada (vía IRT o histórico) cruce con el P(L) actual dentro de la banda.

🤖 LLM como tutores: promesa y trampa

Los LLM aportan lo que los sistemas clásicos no tenían: diálogo abierto, explicación a demanda y generación de ejercicios. Sus modos de fallo son específicos del dominio: **complacencia** (dar la solución cuando el estudiante la pide, destruyendo la práctica), **alucinación pedagógica** (explicaciones seguras e incorrectas, graves justo porque el estudiante no puede detectarlas) y **ausencia de modelo de estudiante** (sin memoria estructurada de qué domina cada quien, el "tutor" repite nivel genérico). Los diseños serios acoplan LLM (interacción) + knowledge tracing (estado) + política de pistas graduadas (pedagogía), y evalúan con **learning gain** (pre/post test), no con satisfacción del usuario.

🧮 Ejemplo trabajado

Habilidad "resta con llevadas": P(L)=0.40, P(S)=0.10, P(G)=0.20, P(T)=0.15. El estudiante responde **correctamente** un ejercicio:

```
Paso 1 — evidencia:
  numerador   = 0.40 × (1−0.10) = 0.36
  denominador = 0.36 + 0.60 × 0.20 = 0.36 + 0.12 = 0.48
  P(L | correcto) = 0.36/0.48 = 0.75

Paso 2 — oportunidad de aprender:
  P(L') = 0.75 + 0.25 × 0.15 = 0.7875
```

Una sola respuesta correcta subió el dominio estimado de 0.40 a 0.79 — mucho, porque P(G)=0.20 hace poco plausible acertar adivinando. Si en cambio hubiera fallado: `P(L|error) = 0.40×0.10 / (0.04 + 0.60×0.80) = 0.04/0.52 ≈ 0.077`, y con la transición `P(L') ≈ 0.077 + 0.923×0.15 ≈ 0.216`. Nótese la asimetría: el error es evidencia fuerte (fallar dominando es raro, P(S)=0.10). Todavía lejos del umbral 0.95: el tutor daría más práctica, no avanzaría de tema.

📊 Propiedades y comparación

Modelo de estudiante	Parámetros	Interpretable	Datos necesarios	Captura prerequisitos	Límite principal
BKT	4 por habilidad	Sí	Pocos	No (habilidades independientes)	Supone habilidad binaria
IRT	Dificultad/discriminación por ítem + habilidad continua	Sí	Medios	No	Estático (no modela aprender)
DKT (RNN)	Miles-millones	No	Muchos	Sí (implícito)	Caja negra; sobreajusta con poco dato
LLM sin estado	—	Narrativa	Cero (pretrained)	Solo en el diálogo	No hay modelo persistente del estudiante

⚠️ Errores conceptuales frecuentes

1. **"Acertó, así que sabe."** Con P(G)=0.25 (opción múltiple de 4), un acierto es evidencia débil; BKT existe precisamente para pesar slip y guess en vez de contar aciertos.
2. **"Personalizar = dejar que el LLM converse."** Sin estado persistente de dominio por habilidad no hay adaptación: la dificultad no cambia con lo aprendido y el tutor repite nivel genérico.

3. **"Cuanta más ayuda, mejor tutor."** Dar la solución elimina la práctica recuperativa; la tutoría eficaz andamia (pistas mínimas crecientes) y tolera el error productivo dentro de la ZPD.
4. **"El learning gain se mide con encuestas de satisfacción."** Los estudiantes prefieren tutores complacientes; el efecto de Bloom se midió con pre/post test. Satisfacción y aprendizaje pueden anticorrelacionar.
5. **"DKT es mejor porque es deep."** Con pocos datos por habilidad, BKT/IRT rinden igual o mejor y son auditables ante estudiantes, docentes y reguladores — en educación la explicabilidad es requisito, no lujo.

🚀 Del aprendizaje a la operación

Un tutor real exige además: un grafo de prerequisitos curado por docentes (la IA no lo inventa sola), calibración de P(S)/P(G) por ítem con datos reales, evaluación con grupos de control y pre/post test antes de afirmar mejora, protección de datos de menores (los historiales de aprendizaje son datos sensibles bajo GDPR/COPPA), y un canal docente: el sistema informa y sugiere, pero la decisión pedagógica de alto impacto (retener, avanzar, derivar) queda en humanos.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("agent")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Corbett, A. T. y Anderson, J. R. (1995). *Knowledge tracing: Modeling the acquisition of procedural knowledge*. UMUAI 4. DOI 10.1007/BF01099821 — uso: fuente primaria del mecanismo estudiado
- Bloom, B. S. (1984). *The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring*. Educational Researcher 13(6). DOI 10.3102/0013189X013006004 — uso: fuente primaria del mecanismo estudiado
- Piech, C. et al. (2015). *Deep Knowledge Tracing*. NeurIPS 2015. arXiv:1506.05908 — uso: fuente primaria del mecanismo estudiado
- VanLehn, K. (2011). *The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems*. Educational Psychologist 46(4). DOI 10.1080/00461520.2011.611369 — uso: fuente primaria del mecanismo estudiado

- Vygotsky, L. S. (1978). *Mind in Society* (zona de desarrollo próximo). Harvard University Press. Ficha editorial — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P141 · El problema de las dos sigmas: buscar instrucción grupal tan eficaz como la tutoría individual	1984	Cuantifica en desviaciones típicas cuánto mejora la tutoría individual sobre la clase convencional, y convierte esa cifra en un problema de ingeniería educativa.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Vygotsky, Lev S. — *Mind in society*	1978	ISBN 9780674576292 · web de la obra	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define ia para educación y aprendizaje adaptativo sin usar una marca o framework como definición.
2. Explica la relación entre education, tutoring, assessment, pedagogy.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?

4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

181 — IA para ciencia, clima y salud responsable

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ia para ciencia, clima y salud responsable** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ia para ciencia, clima y salud responsable usando los conceptos `science`, `climate`, `health`, `validation`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`science`, `climate`, `health`, `validation`

🗺️ Ubicación en el mapa de la IA

Aquí la IA deja de ser producto y vuelve a ser instrumento científico. Tres casos marcan la diferencia entre promesa y resultado: **AlphaFold 2** (2021) resolvió con precisión experimental un problema abierto de 50 años, la predicción de estructura de proteínas; **GraphCast** (2023) superó al modelo determinista operativo del ECMWF en la mayoría de variables meteorológicas a 10 días, en un minuto de cómputo; y la IA clínica acumula, en cambio, una larga lista de modelos publicados que nunca sirvieron a un paciente. Esta clase hereda de aprendizaje profundo, GNN y transformers (partes 4-6) y de la evaluación rigurosa (parte 13): lo que distingue los tres casos no es la arquitectura sino el **régimen de validación**.

📖 Fundamentos

🧬 AlphaFold: por qué funcionó

El problema: dada una secuencia de aminoácidos, predecir la estructura 3D de la proteína. Ingredientes del salto (Jumper et al., 2021, DOI 10.1038/s41586-021-03819-2):

- **Señal evolutiva**: los alineamientos múltiples de secuencias (MSA) contienen co-evolución — residuos que mutan juntos suelen estar en contacto espacial. Es información física escondida en bases de datos públicas.
- **Sesgo inductivo geométrico**: el módulo Evoformer razona sobre pares de residuos y el módulo de estructura predice rotaciones/traslaciones respetando la equivarianza (la física no cambia si giras la molécula).
- **Benchmark ciego**: CASP evalúa contra estructuras experimentales aún no publicadas. AlphaFold 2 alcanzó GDT_TS mediano ~92 en CASP14 (≈ precisión experimental). Ningún resultado autoevaluado habría tenido ese valor probatorio.
- **Incertidumbre calibrada**: pLDDT por residuo indica dónde confiar. Un biólogo usa las regiones con pLDDT alto y descarta las bajas.

🌎 GraphCast y el clima

Los modelos numéricos (NWP) resuelven ecuaciones diferenciales de la atmósfera en supercomputadores. GraphCast (Lam et al., 2023, Science) entrena una GNN sobre 40 años de reanálisis ERA5 para predecir el estado siguiente (paso de 6 h) y se itera autorregresivamente hasta 10 días. Resultado: mejor que HRES del ECMWF en la mayoría de variables/plazos, con inferencia en ~1 minuto en una TPU frente a horas en supercomputador. Matices que la divulgación suele omitir:

- Aprende de **ERA5**, que a su vez es producto de la asimilación de datos del modelo físico: la IA no reemplaza la observación ni la asimilación, se apoya en ellas.
- Es predicción meteorológica (días), **no proyección climática** (décadas bajo escenarios de emisiones): son problemas distintos; extrapolar fuera de la distribución de entrenamiento es exactamente lo que un modelo aprendido no garantiza.
- Los eventos extremos son la cola escasa del dataset: donde más importa acertar es donde menos ejemplos hay.

🏥 Salud: por qué se atasca

La revisión sistemática de Wynants et al. (BMJ 2020) sobre modelos COVID-19 evaluó más de 200 modelos publicados y concluyó que casi todos tenían alto riesgo de sesgo y ninguno era recomendable para uso clínico. Causas recurrentes:

```
1. Fuga de datos / atajos:  el modelo aprende el marcador del hospital o el tipo de
   equipo en la radiografía, no la patología (Zech et al. 2018).
2. Sin validación externa:  entrenar y validar en la misma población; el rendimiento
   cae al cambiar de hospital (distribution shift).
3. Métrica equivocada:  AUC alto con prevalencia baja → precisión inútil (misma
   falacia de tasa base de la clase 179).
4. Sin efecto en desenlaces:  clasificar bien ≠ mejorar la salud del paciente; hace
   falta ensayo prospectivo del sistema sociotécnico completo, no del modelo.
5. Sesgo distribucional:  peor rendimiento en subgrupos poco representados;
   la métrica agregada lo oculta.
```

Marcos como TRIPOD+AI (reporte de modelos predictivos) y SPIRIT/CONSORT-AI (ensayos clínicos con IA) existen porque la comunidad médica ya recorrió este ciclo.

🔬 El patrón común

Los tres dominios comparten el criterio que separa ciencia de demo: **evaluación ciega, prospectiva y externa**, incertidumbre reportada, y un desenlace que le importe a alguien fuera del paper.

🔭 Actualización 2026: agentes científicos y prueba formal

Un resultado reciente llevó ese patrón a matemáticas: un sistema de agentes produjo una solución analítica para Navier–Stokes y una formalización en Lean. La novedad pedagógica no es memorizar el modelo utilizado, sino separar tres capas de evidencia:

1. **Descubrimiento**: miles de agentes exploran y comparten enfoques.
2. **Formalización**: un asistente de pruebas comprueba cada paso contra reglas.
3. **Validación científica**: especialistas independientes revisan enunciado, supuestos, correspondencia entre prueba informal y formal y alcance del resultado.

Lean reduce drásticamente una clase de error lógico, pero no valida por sí solo que se formalizó el teorema correcto ni convierte cualquier salida en ciencia. Además, el emisor reporta un costo aproximado de 10 000 agentes, 130 mil millones de tokens de salida y más de cien horas entre búsqueda y formalización: es un hito de frontera, no una receta eficiente o general. Por eso se registra en `frontier/` como `emerging` y no como paper fundacional.

🧮 Ejemplo trabajado

Un modelo de cribado detecta una condición con prevalencia 1 % en la población cribada. Sensibilidad 95 %, especificidad 90 % ("AUC 0.96" en el paper). Sobre 10,000 personas:

```
Enfermos: 100      Sanos: 9,900
Verdaderos positivos = 0.95 × 100   =    95
Falsos positivos     = 0.10 × 9,900 =   990
Falsos negativos     =                    5

VPP = 95 / (95 + 990) = 95/1,085 ≈ 8.8 %
VPN = 8,910 / (8,910 + 5) ≈ 99.94 %
```

De cada 100 personas con resultado positivo, ~9 tienen la condición y ~91 pasan por pruebas confirmatorias, costo y ansiedad. El modelo puede ser útil (el VPN altísimo sirve para descartar) pero la decisión clínica depende del **costo relativo** de los dos errores y de si existe una prueba confirmatoria barata — nada de eso está en el AUC. Si además el 990 se concentra en un subgrupo demográfico, el daño no es uniforme aunque la métrica agregada no cambie.

📊 Propiedades y comparación

Caso	Señal que explota	Validación decisiva	Estado	Límite honesto
AlphaFold 2	Co-evolución en MSA + geometría	CASP14 (ciego, estructuras no publicadas)	Adoptado; base de datos pública	Estructura estática; complejos, dinámica y efecto de mutaciones siguen abiertos
GraphCast	40 años de ERA5, GNN autorregresiva	Contra HRES/ECMWF en años retenidos	Operativo como complemento	Depende de ERA5; extremos escasos; no es proyección climática
Cribado clínico típico	Correlaciones en historia clínica/imagen	Rara vez externa o prospectiva	Mayoría no llega a la clínica	Fuga, shift, VPP bajo, sin efecto en desenlaces

⚠️ Errores conceptuales frecuentes

1. **"AlphaFold resolvió la biología estructural."** Resolvió la predicción de estructura estática monomérica con precisión útil; dinámica conformacional, complejos y efecto de mutaciones puntuales siguen siendo problemas abiertos.
2. **"Los modelos de IA reemplazan al modelo numérico del clima."** GraphCast se entrena sobre ERA5, producto de la asimilación de datos del sistema físico: coexisten y se necesitan.
3. **"Predecir el tiempo a 10 días valida proyecciones climáticas a 2100."** Son problemas distintos; el segundo exige extrapolar a estados no vistos, justo donde un modelo aprendido no ofrece garantías.
4. **"AUC 0.96 significa que sirve en la clínica."** Con prevalencia 1 %, el VPP puede ser ~9 % (ejemplo trabajado): la utilidad depende de prevalencia, costos asimétricos y prueba confirmatoria.
5. **"Validar en otro conjunto del mismo hospital es validación externa."** No lo es: la fuga por marcadores de equipo y protocolo sobrevive a esa partición; hace falta otra institución y, mejor, otro periodo temporal.

🚀 Del aprendizaje a la operación

Para pasar de este núcleo a un uso real en ciencia, clima o salud faltan: validación externa multi-sitio y prospectiva, reporte según TRIPOD+AI o CONSORT-AI, evaluación desagregada por subgrupos con la métrica que importa clínicamente (VPP/VPN, no solo AUC), monitoreo de *drift* tras el despliegue (los protocolos y equipos cambian), y gobernanza: en salud y clima el modelo informa a un profesional responsable, que es quien decide — la trazabilidad de esa decisión es parte del sistema, no un extra.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- OpenAI (2026). *On the Navier–Stokes Millennium Prize Problem*. openai.com/index/navier-stokes-solution — uso: resultado primario reciente con prueba Lean enlazada; se trata como frontera pendiente de validación independiente
- Jumper, J. et al. (2021). *Highly accurate protein structure prediction with AlphaFold*. Nature 596. DOI 10.1038/s41586-021-03819-2 — uso: fuente primaria del mecanismo estudiado
- Lam, R. et al. (2023). *Learning skillful medium-range global weather forecasting* (GraphCast). Science 382(6677). DOI 10.1126/science.adi2336 — uso: fuente primaria del mecanismo estudiado
- Wynants, L. et al. (2020). *Prediction models for diagnosis and prognosis of covid-19: systematic review and critical appraisal*. BMJ 369:m1328. DOI 10.1136/bmj.m1328 — uso: fuente primaria del mecanismo estudiado
- Zech, J. R. et al. (2018). *Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs*. PLOS Medicine 15(11). DOI 10.1371/journal.pmed.1002683 — uso: fuente primaria del mecanismo estudiado
- Collins, G. S. et al. (2024). *TRIPOD+AI statement: updated guidance for reporting clinical prediction models*. BMJ 385:e078378. DOI 10.1136/bmj-2023-078378 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P47 · Predicción de estructura de proteínas de alta precisión con AlphaFold	2021	Resuelve en la práctica un problema abierto de cincuenta años en biología, y demuestra que la IA puede producir conocimiento científico, no solo productos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define ia para ciencia, clima y salud responsable sin usar una marca o framework como definición.
2. Explica la relación entre science, climate, health, validation.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

182 — Cómo vigilar la frontera sin perseguir modas

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **cómo vigilar la frontera sin perseguir modas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cómo vigilar la frontera sin perseguir modas usando los conceptos `frontier`, `evidence`, `maturity`, `hype`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`frontier`, `evidence`, `maturity`, `hype`

🗺️ Ubicación en el mapa de la IA

Esta clase no añade una técnica: añade el método para decidir qué técnicas merecen tu tiempo. Después de 181 clases, el riesgo profesional ya no es no saber, sino gastar atención en lo que caducará en seis meses mientras se descuida lo que lleva décadas siendo cierto (álgebra lineal, probabilidad, búsqueda, evaluación). Es la contrapartida metodológica de todo el programa y el prerequisito del capstone: un sistema evolutivo necesita un criterio explícito de qué incorporar y cuándo.

📖 Fundamentos

🧱 Núcleo estable vs frontera

Divide todo lo que aprendes en dos capas con vidas medias distintas:

```
NÚCLEO ESTABLE (vida media: décadas)
  probabilidad y estadística · optimización · complejidad · búsqueda
  teoría del aprendizaje · evaluación experimental · ingeniería de software

FRONTERA (vida media: meses-años)
  APIs concretas · nombres de modelos · frameworks de agentes · benchmarks de moda
```

Regla de asignación de tiempo: el núcleo se estudia en profundidad porque se amortiza; la frontera se **vigila** con un proceso barato y se adopta solo cuando cruza un umbral. Invertir al revés es la definición operativa de perseguir modas.

📏 Criterios de madurez

Un tema puede clasificarse con evidencia observable, no con entusiasmo:

Nivel	Señales objetivas	Qué hacer
`emerging`	1-2 papers o un anuncio; sin implementaciones independientes; sin evaluación externa	Anotar y esperar; leer el abstract, no reescribir tu stack
`current`	Implementaciones múltiples, uso reportado fuera de la organización que lo creó, evaluaciones de terceros	Prototipo acotado con criterio de salida
`standard`	Especificación pública estable, adopción multi-proveedor, herramientas y documentación maduras	Adoptar si resuelve un problema que tienes
`deprecated`	Sustituido, sin mantenimiento, o refutado	Migrar y documentar por qué

🔍 Señal vs ruido: preguntas de filtrado

Ante cualquier anuncio, cinco preguntas ordenadas por poder de descarte:

1. **¿Contra qué se comparó?** Sin baseline explícito y fuerte, no hay resultado.
2. **¿Quién evaluó?** Autoevaluación del proveedor ≠ evaluación independiente ≠ benchmark ciego (CASP, competiciones con test oculto). Cada escalón vale más.
3. **¿Hay contaminación?** Si el benchmark es público y anterior al corte de entrenamiento, la métrica puede estar memorizada.
4. **¿Es reproducible?** Pesos, código, datos y semilla, o al menos una receta que un tercero haya seguido con éxito.
5. **¿Qué costo tiene?** Un resultado que exige 1000× el cómputo por 2 % de mejora es un resultado sobre escalado, no sobre el método.

Sesgos que atacan estas preguntas: **survivorship bias** (solo se publican los éxitos), **publicación selectiva de demos** (el video muestra el caso que funcionó), y el ciclo de expectativas: la novedad se sobreestima a corto plazo y su efecto real se subestima a largo plazo.

🗂️ Vigilancia como proceso (el caso de este repositorio)

El propio repositorio implementa el patrón: el directorio `frontier/` mantiene `current-topics.yaml`, un registro donde cada tema declara `id`, `name`, `category`, `maturity`, `reviewed` (fecha) y `source` (enlace verificable) más una `reason` de por qué está ahí. El diseño tiene tres propiedades importantes:

- **Separación física**: la frontera vive fuera de las 184 clases, así que su caducidad no contamina el núcleo. El README de `frontier/` lo dice explícitamente: no sustituyen el núcleo estable.
- **Fecha de revisión obligatoria**: un tema sin `reviewed` reciente es una afirmación sin respaldo temporal; el campo convierte la obsolescencia en algo detectable por script.
- **Fuente obligatoria**: cada entrada apunta a documentación primaria, de modo que actualizar un tema es re-leer la fuente, no recordar el rumor.

Un ciclo de vigilancia razonable: revisar el registro cada 4-8 semanas, mover temas entre niveles con justificación escrita, y borrar los que no sobrevivieron — mantener un registro honesto de lo que NO cuajó es tan valioso como la lista de lo adoptado.

🧮 Ejemplo trabajado

Llegan cuatro anuncios la misma semana. Aplicamos la rúbrica (2 puntos por criterio cumplido, 1 parcial, 0 ausente) sobre: baseline, evaluador independiente, reproducibilidad, costo declarado.

```
Tema                          base  indep  repro  costo   total  → nivel
A. Protocolo con spec pública
   y 3 implementaciones          2      2      2      2     8/8  → standard (adoptar si aplica)
B. Modelo con +2 % en un
   benchmark público, autoeval   1      0      1      0     2/8  → emerging (anotar)
C. Framework de agentes nuevo,
   usado por 2 empresas ajenas,
   sin evaluación comparativa    0      1      2      1     4/8  → current (prototipo acotado)
D. Resultado con 1000× cómputo
   y +2 %, código cerrado        2      1      0      2     5/8  → current, pero el
                                                                   hallazgo es sobre escalado
```

Decisión: adoptar A si resuelve un problema existente; prototipar C con criterio de salida escrito ("si en 2 semanas no reduce X, se descarta"); anotar B con fecha y revisarlo en 8 semanas; archivar D como evidencia de que la mejora depende del presupuesto, no del método. Ninguna de las cuatro decisiones exige haber leído los cuatro papers completos: el filtro es barato por diseño.

📊 Propiedades y comparación

Estrategia de vigilancia	Costo de atención	Riesgo de perder algo importante	Riesgo de perseguir modas
Ignorar la frontera	Nulo	Alto (obsolescencia silenciosa)	Nulo
Leer todo el feed diario	Muy alto	Bajo	Muy alto
Adoptar lo que hace el competidor	Bajo	Medio	Alto (copias su error también)
Registro con madurez + revisión periódica	Bajo-medio	Bajo	Bajo

⚠️ Errores conceptuales frecuentes

1. **"Estar al día = leer todo lo que sale."** El feed crece más rápido que cualquier atención humana; sin criterio de descarte, leer más produce menos decisiones buenas, no más.
2. **"Si un laboratorio grande lo publica, está validado."** El prestigio del emisor no sustituye baseline, evaluación independiente ni reproducibilidad; son preguntas sobre el resultado, no sobre quién lo firma.
3. **"Un benchmark superado significa capacidad general."** El benchmark define lo que se midió; la contaminación y el sobreajuste al test son la norma, no la excepción, cuando el conjunto es público.
4. **"Adoptar temprano da ventaja."** Da ventaja *si* el tema sobrevive; el costo esperado incluye las migraciones de todo lo que no sobrevivió. Por eso el prototipo acotado con criterio de salida domina a la adopción entusiasta.
5. **"La frontera y el núcleo compiten por el mismo tiempo."** Compiten solo si se estudian igual: el núcleo se aprende en profundidad, la frontera se registra en minutos. Confundir los dos modos es lo que produce la sensación de no llegar a nada.

🚀 Del aprendizaje a la operación

Llevar esto a un equipo real exige: un dueño del registro (si es de todos, no se revisa), una cadencia agendada de revisión, un formato con campos obligatorios —fuente, fecha, madurez, razón— que haga detectable la obsolescencia por script, un presupuesto explícito de experimentación (p. ej. 10 % del tiempo) con criterios de salida escritos ANTES de empezar, y la disciplina de registrar también los descartes: sin memoria de lo que no funcionó, el equipo repetirá el mismo prototipo cada dos años con otro nombre.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;

- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Directorio `frontier/` de este repositorio: `frontier/README.md` y `frontier/current-topics.yaml` — registro con `maturity`, `reviewed` y `source` por tema.
- Stanford HAI (anual). *AI Index Report* — datos longitudinales sobre capacidades, cómputo y adopción. aiindex.stanford.edu — uso: referencia consultada en su fuente original
- Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 2015. PDF NeurIPS — uso: referencia consultada en su fuente original
- Lipton, Z. C. y Steinhardt, J. (2019). *Troubling Trends in Machine Learning Scholarship*. arXiv:1807.03341 — uso: fuente primaria del mecanismo estudiado
- Ioannidis, J. P. A. (2005). *Why Most Published Research Findings Are False*. PLOS Medicine 2(8). DOI 10.1371/journal.pmed.0020124 — uso: fuente primaria del mecanismo estudiado
- Kapoor, S. y Narayanan, A. (2023). *Leakage and the reproducibility crisis in machine-learning-based science*. Patterns 4(9). DOI 10.1016/j.patter.2023.100804 — uso: fuente primaria del mecanismo estudiado

---

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

📝 Evaluación completa

❓ Preguntas

1. Define cómo vigilar la frontera sin perseguir modas sin usar una marca o framework como definición.
2. Explica la relación entre frontier, evidence, maturity, hype.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

183 — Inferencia privada, computación confidencial y attestation

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 6
Laboratorio: `confidential_inference` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Diseñar un flujo de inferencia que proteja datos **mientras se procesan**, verificar la identidad del entorno antes de entregar secretos y explicar con precisión qué garantías aporta —y cuáles no— la computación confidencial.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir cifrado en tránsito, en reposo y en uso.
2. Explicar TEE, medición de runtime, remote attestation y liberación condicionada de claves.
3. Diseñar una política verificable para inferencia privada y memoria cifrada.
4. Comparar TEE con ejecución local, ZDR, privacidad diferencial, federación, MPC y FHE.
5. Construir un threat model que incluya host, side channels, metadatos, salidas y supply chain.

🧩 Conceptos centrales

`confidential computing`, `private inference`, `TEE`, `remote attestation`, `data in use`, `key release`

🗺️ Ubicación en el mapa de la IA

Las clases 165 y 177 reducen qué datos entran, cuánto se retiene y qué puede inferirse estadísticamente. Sin embargo, un servicio aún debe descifrar entradas para ejecutar el modelo. La computación confidencial añade un perímetro para **datos en uso**: intenta que el plaintext y las claves solo existan dentro de un entorno aislado cuya identidad pueda comprobar el cliente o un servicio de claves.

Esta arquitectura ganó relevancia con asistentes persistentes y memoria entre dispositivos. Es una nueva frontera de sistemas y confianza, no una promesa de que el modelo sea privado, correcto o seguro por definición.

📖 Fundamentos

🔐 Tres estados del dato

```
en tránsito  -> TLS protege el canal
en reposo    -> cifrado de disco/objeto protege almacenamiento
en uso       -> un entorno de ejecución aislado protege mientras se calcula
```

TLS no impide que el servidor vea el dato después de descifrarlo. El cifrado de disco tampoco protege una página de memoria ya abierta por el proceso. Un **Trusted Execution Environment (TEE)** busca aislar memoria y ejecución incluso frente al sistema operativo o hipervisor anfitrión, dentro del threat model declarado por su plataforma.

📜 Measurement y remote attestation

Un measurement es una huella criptográfica del código, configuración y estado inicial que arrancó dentro del entorno. La **remote attestation** entrega evidencia firmada para que un verificador remoto contraste esa huella con una allowlist.

```
cliente -> nonce de frescura
TEE     -> evidencia firmada + measurement + configuración
verificador -> firma válida? nonce nuevo? runtime aprobado? debug desactivado?
KMS     -> libera clave SOLO si la política completa pasa
```

La attestation responde «¿arrancó este runtime aprobado bajo estas condiciones?». No responde «¿el modelo dará una respuesta correcta?» ni «¿el hardware carece de fallos?». La arquitectura RATS del IETF separa explícitamente Attester, Verifier y Relying Party; esa separación evita que el mismo componente se auto-certifique.

🧠 Inferencia privada y memoria persistente

Un pipeline mínimo cifra la petición para una clave ligada al entorno, verifica la attestation, procesa dentro del perímetro y cifra la respuesta antes de salir. Si conserva memoria, cada objeto debe quedar cifrado con claves que el host no controle y con políticas de borrado, revocación y versión.

Zero Data Retention (ZDR) es una política operacional de no conservar datos; un TEE es un control técnico sobre el procesamiento. Son controles complementarios y deben verificarse por separado. La afirmación comercial «private» no sustituye evidencia sobre measurement, gestión de claves, logs, metadatos y operadores de confianza.

🧱 Threat model: lo que queda fuera

Una attestation válida todavía puede coexistir con:

- un modelo que memoriza o revela información en su salida;
- prompt injection y tools con permisos excesivos;
- side channels de tiempo, caché, tamaño o tráfico;
- firmware, silicio o autoridad de attestation comprometidos;
- imagen aprobada pero vulnerable o dependencias maliciosas;
- logs, billing y metadatos fuera del enclave;
- cliente comprometido antes de cifrar o después de descifrar.

Por eso el control correcto es **attestation + minimización + política de retención + seguridad de aplicación + auditoría**, no «usa TEE» como casilla aislada.

🧮 Ejemplo trabajado

Una organización quiere resumir expedientes sensibles sin entregarlos en claro al host:

1. Publica una política con measurement aprobado, región, `debug=false` y `ZDR=true`.
2. El cliente genera un nonce y solicita evidencia fresca.
3. Un verificador independiente valida firma, nonce y cadena de certificados.
4. Compara measurement y configuración con la política versionada.
5. Solo entonces el KMS envuelve una clave para ese entorno.
6. El runtime descifra, infiere y cifra la respuesta dentro del perímetro.
7. Logs externos contienen IDs y estados, nunca prompt, respuesta o clave.
8. Si cambia la imagen, la medición deja de coincidir y la liberación se bloquea hasta aprobar explícitamente la nueva versión.

La decisión es *fail closed*: una evidencia ausente, vencida o inesperada no degrada a «continuar sin privacidad».

📊 Comparación de controles

Técnica	Protege principalmente	Confianza residual	No resuelve
Ejecución local	datos fuera de la nube	dispositivo y software local	capacidad limitada, cliente comprometido
TEE + attestation	datos en uso en infraestructura remota	hardware, firmware, autoridad y runtime aprobado	corrección del modelo, side channels completos
ZDR	retención operacional	implementación y auditoría del proveedor	acceso durante procesamiento
Privacidad diferencial	contribución individual en un mecanismo	calibración y contabilidad de ε	secreto exacto de una petición concreta
Aprendizaje federado	localización del dato de entrenamiento	clientes, agregación y protocolo	fuga por gradientes
MPC/FHE	cómputo distribuido o cifrado	protocolo, claves y coste operacional	latencia y soporte general eficiente

⚠️ Errores conceptuales frecuentes

1. **«TLS ya protege la inferencia».** Protege el trayecto, no el dato descifrado en memoria.
2. **«Attestation demuestra que el modelo es seguro».** Demuestra identidad y configuración inicial dentro de un modelo de amenaza; no calidad, alineamiento ni ausencia de bugs.
3. **«ZDR y TEE son sinónimos».** Una es política de retención; el otro, aislamiento de ejecución.
4. **«El proveedor no puede ver nada».** Metadatos, tamaños, tiempos o salidas pueden quedar fuera.
5. **«Un measurement aprobado puede durar para siempre».** Parches y revocaciones exigen ciclo de vida, expiración y nueva aprobación.

🚀 Del aprendizaje a la operación

En producción: inventario de datos y confianza; política de attestation versionada; nonces y protección contra replay; verificador separado; liberación de claves condicionada; allowlist de measurements; revocación; actualización segura; logs sin contenido; pruebas negativas; SLO de verificación; procedimiento fail-closed y evidencia de terceros sobre el hardware cuando exista.

🧪 Laboratorio

```
python lab.py
```

El laboratorio compara una evidencia aprobada con otra que usa una imagen desconocida y modo debug. Simula la decisión de liberar una clave, pero **no ejecuta hardware confidencial**.

🔍 Evidencia esperada

- política y measurement esperado;
- decisión por solicitud y razones de rechazo;
- liberación condicionada de clave;
- garantías declaradas y propiedades fuera de alcance;
- limitaciones explícitas de la simulación.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia.

📝 Evaluación

Criterio	Peso
Threat model y límites	30 %
Política y attestation	30 %
Pruebas negativas y fail-closed	25 %
Evidencia y comunicación	15 %

Consulta assessment.md para preguntas y criterio de aceptación.

❓ Preguntas frecuentes

¿Necesito hardware TEE para completar la clase?
No. El laboratorio enseña el contrato verificable y declara que la evidencia es simulada.

¿La inferencia confidencial reemplaza la minimización?
No. Todo control puede fallar; enviar menos datos sigue reduciendo impacto.

¿Private inference equivale a privacidad perfecta?
No. El nombre del producto no define el threat model ni las garantías demostradas.

🔗 Referencias

- IETF RFC 9334, *Remote ATtestation procedureS (RATS) Architecture* — uso: estándar técnico para separar Attester, Verifier y Relying Party
- Confidential Computing Consortium, *About Confidential Computing* — uso: definición sectorial de protección de datos en uso
- Google DeepMind (2026), *Advancing Private AI Compute with secure, server-side memory* — uso: arquitectura empresarial reportada; no se trata como validación independiente
- OpenAI (2026), *DevDay 2026 Recap* — uso: anuncio empresarial de Private Inference; se registra como preview, no como capacidad auditada aquí

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 14 · capítulo sobre el futuro de la IA
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

Normas y documentación oficial que aplica esta clase: Remote ATtestation procedureS (RATS) Architecture

📝 Evaluación completa

❓ Preguntas

1. Distingue cifrado en tránsito, en reposo y **en uso**.
2. Explica por qué una medición de runtime y una attestation no prueban que el modelo sea correcto.
3. Separa las garantías de un TEE de las de privacidad diferencial, aprendizaje federado y ZDR.
4. ¿Qué debe comprobar una política antes de liberar una clave a un entorno confidencial?
5. Propón dos ataques que queden fuera del perímetro de una attestation válida.

🏆 Reto verificable

Amplía el laboratorio con una tercera solicitud que tenga runtime aprobado, pero región incorrecta o retención activa. Demuestra que la decisión sigue siendo `deny` y conserva:

- el hash de la política aplicada;
- cada razón de rechazo;
- la versión del verificador;
- las garantías y límites declarados.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] Ninguna clave se libera antes de verificar attestation y política.
- [ ] La conclusión no presenta la simulación como ejecución en un TEE real.
- [ ] El threat model incluye host, side channels, salida del modelo y cadena de suministro.

---

184 — Capstone final: sistema de IA evolutivo

Parte: 14 — Frontera, investigación y proyectos integradores
Nivel: frontera · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **capstone final: sistema de ia evolutivo** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone final: sistema de ia evolutivo usando los conceptos `portfolio`, `system`, `agents`, `safety`, `operations`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`portfolio`, `system`, `agents`, `safety`, `operations`

🗺️ Ubicación en el mapa de la IA

Esta clase cierra el programa integrando las 15 partes en un solo sistema. No introduce una técnica nueva: obliga a componer las que ya conoces bajo restricciones reales —evidencia, seguridad, costo, operación— y a defender cada decisión. Un "sistema de IA evolutivo" es aquel que puede mejorar con el uso sin degradarse en silencio: combina un núcleo determinista auditable, componentes aprendidos, memoria y recuperación, agentes con herramientas, evaluación continua y un proceso de gobierno del cambio. Es el punto donde el programa deja de enseñar piezas y empieza a exigir arquitectura.

📖 Fundamentos

🧭 Qué significa "evolutivo"

Tres propiedades, en orden de dificultad:

1. **Mejorable**: existe un canal por el que la experiencia se convierte en cambio (datos etiquetados por el uso, prompts versionados, políticas ajustadas, índices actualizados).

2. **Verificable**: cada cambio se acepta o rechaza contra una suite de evaluación antes de llegar a los usuarios. Sin esto, "evolutivo" significa "deriva".
3. **Reversible**: todo cambio tiene un artefacto versionado y un camino de rollback. La capacidad de deshacer es lo que hace segura la capacidad de cambiar.

🏗️ Arquitectura de referencia (mapa de las 15 partes)

```
[Entrada del usuario]
   ↓
GUARDIAS DE ENTRADA          ← parte 13: validación, política, datos ≠ instrucciones
   ↓
ORQUESTADOR / AGENTE         ← partes 9-10: planificar, llamar herramientas, delegar
   ├─ RECUPERACIÓN + MEMORIA ← parte 8: índice, contexto, citas verificables
   ├─ MODELO(S)              ← partes 4-7: LLM, visión, generación; ajuste y prompting
   ├─ NÚCLEO DETERMINISTA    ← partes 1-3: búsqueda, reglas, probabilidad, decisión
   └─ HERRAMIENTAS           ← partes 11-12: efectos en el mundo, APIs, ejecución
   ↓
GUARDIAS DE SALIDA           ← parte 13: verificación de afirmaciones, límites, PII
   ↓
[Respuesta + evidencia citada]        →  TELEMETRÍA → EVALUACIÓN → cambio versionado
                                                    ↑ partes 12-13 (bucle evolutivo)
```

La regla de composición más importante: **cuanto más determinista sea la parte que toma la decisión final, más auditable es el sistema**. Los componentes aprendidos proponen; el núcleo determinista y el humano disponen, y ambos dejan traza.

📋 El contrato de evidencia

Todo el programa se apoya en un patrón único, visible en cada laboratorio: un resultado no es un texto, es una estructura con `kind` (qué se ejecutó), `evidence` (hechos inspeccionables) y `limitations` (lo que este resultado NO permite concluir), más la semilla y la configuración que lo hacen reproducible. Un capstone que produce una demo impresionante sin ese contrato no es un sistema: es una anécdota.

⚖️ Los cuatro trade-offs que estructuran el diseño

```
Autonomía  ⇔  Control       más pasos autónomos = más valor y más radio de daño
Capacidad  ⇔  Costo         test-time compute mejora resultados y multiplica el gasto
Recuerdo   ⇔  Privacidad    memoria útil = datos retenidos = superficie regulatoria
Novedad    ⇔  Estabilidad   adoptar lo nuevo vs mantener lo que ya está validado
```

Ninguno tiene solución óptima general: el diseño consiste en elegir un punto, declararlo y medirlo. Un capstone se evalúa por la calidad de esas decisiones declaradas, no por la ausencia de compromisos.

🚨 Qué convierte un prototipo en sistema

- **Un caso de uso acotado** con un desenlace medible (no "asistente de todo").
- **Baseline no-IA**: qué haría una regla, una búsqueda o un humano; sin él no se puede afirmar mejora.
- **Suite de evaluación** con casos de éxito, casos límite y casos adversariales, ejecutable en CI.
- **Límites explícitos** y un plan de qué hacer cuando el sistema no sabe (rechazar y escalar es una respuesta válida y frecuentemente la correcta).
- **Operación**: telemetría, costos, alertas de regresión y dueño responsable.

🧮 Ejemplo trabajado

Diseño mínimo de un capstone: *asistente de consultas sobre la normativa interna de una empresa*. Se recorre el mapa decidiendo y declarando:

```
1. CASO Y DESENLACE
   Responder preguntas de normativa con cita al documento fuente.
   Métrica: % de respuestas con cita correcta verificada por muestreo humano.
   Baseline no-IA: buscador por palabras clave sobre el mismo corpus.

2. COMPOSICIÓN
   Recuperación (parte 8) → LLM que responde SOLO con lo recuperado (parte 6)
   → verificador determinista: si la respuesta no cita un documento del índice,
     se rechaza y devuelve "no encontrado" (partes 1-3, decisión final determinista).

3. GUARDIAS
   Entrada: el texto del usuario nunca se interpreta como instrucción de sistema.
   Salida: se bloquea toda respuesta sin cita y toda que incluya datos personales.

4. EVALUACIÓN (suite en CI)
   30 preguntas con respuesta conocida  → exactitud y cita correcta
   10 preguntas fuera del corpus        → debe responder "no encontrado" (¡clave!)
   10 intentos de inyección de prompt   → no debe cambiar de rol ni filtrar el prompt

5. CONTRATO DE SALIDA
   {kind, answer, evidence: [doc_id, sección, fragmento], limitations, seed, version}

6. TRADE-OFFS DECLARADOS
   Autonomía baja (no ejecuta acciones, solo responde) · costo acotado (1 llamada +
   recuperación) · sin memoria de usuario (privacidad sobre personalización) ·
   modelo fijado por versión (estabilidad sobre novedad).

7. EVOLUCIÓN
   Cada pregunta con cita marcada como incorrecta entra al conjunto de evaluación.
   Un cambio de prompt, índice o modelo se promueve solo si la suite no regresa.
```

Ese documento de siete puntos —no el código— es el entregable que distingue un capstone evaluable de una demo.

📊 Propiedades y comparación

Nivel de madurez del capstone	Qué demuestra	Evidencia mínima	Riesgo dominante
Demo	Que algo corre	Captura o video	Confundir "funciona una vez" con "funciona"
Prototipo reproducible	Que corre igual mañana	Semilla, versión, contrato JSON	Sin baseline: no se sabe si aporta
Sistema evaluado	Que es mejor que el baseline	Suite de evaluación + comparación	Sobreajuste a la propia suite
Sistema operable	Que puede vivir en producción	Telemetría, costos, rollback, dueño	Degradación silenciosa por drift

⚠️ Errores conceptuales frecuentes

1. **"El capstone es el código."** El entregable central es el documento de decisiones: caso acotado, baseline, composición, guardias, suite y límites. El código sin ese marco no es evaluable.
2. **"Más componentes = mejor sistema."** Cada componente aprendido añade una fuente de error y de costo. La arquitectura fuerte usa el componente determinista más simple que resuelva cada subproblema.
3. **"Evolutivo significa que se reentrena solo."** Significa que existe un canal de mejora **verificada y reversible**. Sin suite de evaluación y rollback, la automejora automática es deriva no supervisada.
4. **"Si la demo impresiona, el sistema funciona."** La demo se elige entre los casos que salieron bien (survivorship bias). Lo que informa es el comportamiento en casos límite: fuera de corpus, entradas adversariales, y el caso de "no sé".
5. **"Rechazar responder es un fallo."** Es una capacidad: un sistema que reconoce sus límites y escala al humano es más útil y mucho más seguro que uno que siempre responde con confianza uniforme.

🚀 Del aprendizaje a la operación

Lo que separa este capstone de un sistema real y debes declarar explícitamente: datos reales con su gobernanza (permisos, retención, PII), pruebas con usuarios que no lo construyeron, presupuesto y alertas de costo por consulta, monitoreo de regresiones y de drift tras cada cambio, un plan de incidentes con dueño y camino de rollback, y revisión legal si el dominio está regulado. La conclusión honesta de un capstone termina con "esto es lo que demostré" y "esto es exactamente lo que faltaría para producción" — y esa segunda lista es la que prueba que aprendiste el programa.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;

- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 2015. PDF NeurIPS — uso: referencia consultada en su fuente original
- Russell, S. y Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4.ª ed., cap. 2 (agentes y entornos) y cap. 27 (futuro de la IA). aima.cs.berkeley.edu — uso: desarrollo extendido del tema
- NIST (2023). *AI Risk Management Framework 1.0* — funciones Govern, Map, Measure, Manage. DOI 10.6028/NIST.AI.100-1 — uso: fuente primaria del mecanismo estudiado
- Breck, E. et al. (2017). *The ML Test Score: A Rubric for ML Production Readiness*. IEEE Big Data 2017. DOI 10.1109/BigData.2017.8258038 — uso: fuente primaria del mecanismo estudiado
- Model Context Protocol — especificación oficial de conexión entre aplicaciones de IA y herramientas. modelcontextprotocol.io — uso: marco normativo de referencia
- Amershi, S. et al. (2019). *Software Engineering for Machine Learning: A Case Study*. ICSE-SEIP 2019. DOI 10.1109/ICSE-SEIP.2019.00042 — uso: fuente primaria del mecanismo estudiado

---

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 2 · obra de referencia de la parte 14
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	obra de referencia de la parte 14 · límites de los métodos actuales

Normas y documentación oficial que aplica esta clase: Model Context Protocol

📝 Evaluación completa

❓ Preguntas

1. Define capstone final: sistema de ia evolutivo sin usar una marca o framework como definición.
2. Explica la relación entre portfolio, system, agents, safety, operations.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
